

Slip 1

Q.1) Write a C Menu driven Program to implement following functionality

a) Accept Available

b) Display Allocation, Max

c) Display the contents of need matrix

d) Display Available

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	2	3	2	9	7	5	3	3	2
P1	4	0	0	5	2	2			
P2	5	0	4	1	0	4			
P3	4	3	3	4	4	4			
P4	2	2	4	6	5	5			

```
#include<stdio.h>
```

```
#define MAX 10
```

```
int m,n,avail[MAX],alloc[MAX][MAX],
```

```
max[MAX][MAX],need[MAX][MAX];
```

```
void accept()
```

```
{
```

```
int i,j;
```

```
printf("Enter no.of process:");
```

```
scanf("%d",&n);
```

```
printf("Enter no.of resource types:");
```

```
scanf("%d",&m);
```

```
printf("Enter no.of allocated resources of each resource type by each process:\n");
```

```
for(i=0;i<n;i++)
```

```
{
```

```
printf("P%d:\n",i);
```

```
for(j=0;j<m;j++)
```

```
{
```

```
printf("%c:",65+j);
```

```
scanf("%d",&alloc[i][j]);
```

```

}
}

printf("Enter no.of maximum resources of each resource type by each process:\n");
for(i=0;i<n;i++)
{
printf("P%d:\n",i);
for(j=0;j<m;j++)
{
printf("%c:",65+j);
scanf("%d",&max[i][j]);
}
}
}

void calc_need()
{
int i,j;
for(i=0;i<n;i++)
for(j=0;j<m;j++)
need[i][j]=max[i][j]-alloc[i][j];
}

void printt()
{
int i,j;
printf("\tAllocation\tMax\tNeed\n\t");
for(i=0;i<3;i++)
{
for(j=0;j<m;j++)
printf("%3c",65+j);
printf("\t");
}
printf("\n");

```

```

for(i=0;i<n;i++)
{
printf("P%d\t",i);
for(j=0;j<m;j++)
printf("%3d",alloc[i][j]);
printf("\t");
for(j=0;j<m;j++)
printf("%3d",max[i][j]);
printf("\t");
for(j=0;j<m;j++)
printf("%3d",need[i][j]);
printf("\n");
}
}

int main()
{
int i,j,pno;
accept();
int choice;
do{
printf("\n Press 1 Enter availble matrix \n Press 2 to Display allocation and max ,need \n Press 3 to
Display Available \n press 4 to Exit ");
printf("\n Enter your choice:");
scanf("%d", &choice);
switch (choice)
{
case 1:
printf("Enter Available:\n");
for(i=0;i<m;i++)
{
scanf("%d",&avail[i]);

```

```

        }
        break;
case 2:
    calc_need();
    printt();
    break;
case 3:
    printf("Available\n");
    for(j=0;j<m;j++)
        printf("%3d",avail[j]);
    printf("\t");
    printf("\n");
    break;
exit(1);
default:
    printf("\nInvalid choice\n");
    break;
}
}
while(choice!=0);
return 0;
}

```

Q.2) Write a C program that behaves like a shell which displays the command prompt '\$'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'count' as

- a. \$ count c filename: It will display the number of characters in given file**
- b. \$ count w filename: It will display the number of words in given file**
- c. \$ count l filename: It will display the number of lines in given file**

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```

#include <fcntl.h>

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/wait.h>

void make_toks(char *s, char *tok[])

{

    int i=0;

    char *p;

    p = strtok(s, " ");

    while(p!=NULL)

    {

        tok[i++]=p;

        p=strtok(NULL, " ");

    }

    tok[i]=NULL;

}

void count(char *fn, char op)

{

    int fh,cc=0,wc=0,lc=0;

    char c;

    fh = open(fn,O_RDONLY);

    if(fh==-1)

    {

        printf("File %s not found.\n",fn);

        return;

    }

    while(read(fh,&c,1)>0)

    {

        if(c==' ') wc++;

```

```
else if(c=='\n')
{
wc++;
lc++;
}
cc++;
}
close(fh);
switch(op)
{
case 'c':
printf("No.of characters:%d\n",cc-1);
break;
case 'w':
printf("No.of words:%d\n",wc);
break;
case 'l':
printf("No.of lines:%d\n",lc+1);
break;
}
}
int main()
{
char buff[80],*args[10];
int pid;
while(1)
{
printf("myshell$ ");
fflush(stdin);
fgets(buff,80,stdin);
buff[strlen(buff)-1]='\0';
```

```

make_toks(buff,args);

if(strcmp(args[0],"count")==0)

count(args[2],args[1][0]);

else

{

pid = fork();

if(pid>0)

wait();

else

{

if(execvp(args[0],args)==-1)

printf("Bad command.\n");

}

}

}

return 0;

}

```

Slip 2

Q.1) Write a program to simulate Linked file allocation method. Assume disk with 'n' number of blocks. Give value of 'n' as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned below and implement each option.

- **Show Bit Vector**
- **Create New File**
- **Show Directory**
- **Exit**

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#define MAX 200
typedef struct dir
{
char fname[20];
int start;
struct dir *next;
}NODE;
NODE *first,*last;
int n,fb,bit[MAX];

```

```

void init()
{
    int i;
    printf("Enter total no.of disk blocks:");
    scanf("%d",&n);
    fb = n;
    for(i=0;i<10;i++)
    {
        int k = rand()%n;
        if(bit[k]!=-2)
        {
            bit[k]=-2;
            fb--;
        }
    }
}

void show_bitvector()
{
    int i;
    for(i=0;i<n;i++)
        printf("%d ",bit[i]);
    printf("\n");
}

void show_dir()
{
    NODE *p;
    int i;
    printf("File\tChain\n");
    p = first;
    while(p!=NULL)
    {
        printf("%s\t",p->fname);
        i = p->start;
        while(i!=-1)
        {
            printf("%d->",i);
            i=bit[i];
        }
        printf("NULL\n");
        p=p->next;
    }
}

void create()
{
    NODE *p;
    char fname[20];
    int i,j,nob;
    printf("Enter file name:");

```

```

scanf("%s",fname);
printf("Enter no.of blocks:");
scanf("%d",&nob);
if(nob>fb)
{
printf("Failed to create file %s\n",fname);
return;
}
for(i=0;i<n;i++)
{
if(bit[i]==0) break;
}
p = (NODE*)malloc(sizeof(NODE));
strcpy(p->fname,fname);
p->start=i;
p->next=NULL;
if(first==NULL)
first=p;
else
last->next=p;
last=p;
fb-=nob;
j=i+1;
nob--;
while(nob>0)
{
if(bit[j]==0)
{
bit[i]=j;
i=j;
nob--;
}
j++;
}
bit[i]=-1;
printf("File %s created successully.\n",fname);
}

void delete()
{
char fname[20];
NODE *p,*q;
int nob=0,i,j;
printf("Enter file name to be deleted:");
scanf("%s",fname);
p = q = first;
while(p!=NULL)
{
if(strcmp(p->fname,fname)==0)

```

```

break;
q=p;
p=p->next;
}
if(p==NULL)
{
printf("File %s not found.\n",fname);
return;
}
i = p->start;
while(i!=-1)
{
nob++;
j = i;
i = bit[i];
bit[j] = 0;
}
fb+=nob;
if(p==first)
first=first->next;
else if(p==last)
{
last=q;
last->next=NULL;
}
else
q->next = p->next;
free(p);
printf("File %s deleted successfully.\n",fname);
}
int main()
{
int ch;
init();
while(1)
{
printf("1.Show bit vector\n");
printf("2.Create new file\n");
printf("3.Show directory\n");
printf("4.Delete file\n");
printf("5.Exit\n");
printf("Enter your choice (1-5):");
scanf("%d",&ch);
switch(ch)
{
case 1:
show_bitvector();
break;

```

```

case 2:
create();
break;
case 3:
show_dir();
break;
case 4:
delete();
break;
case 5:
exit(0);
}
}
return 0;
}

```

Q.2) Write a simulation program for disk scheduling using FCFS algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments. 55, 58, 39, 18, 90, 160, 150, 38, 184

Start Head Position: 50

```

#include<stdio.h>
#include<stdlib.h>
int main()
{
int RQ[100],i,n,TotalHeadMoment=0,initial;
printf("Enter the number of Requests\n");
scanf("%d",&n);
printf("Enter the Requests sequence\n");
for(i=0;i<n;i++)
scanf("%d",&RQ[i]);
printf("Enter initial head position\n");
scanf("%d",&initial);
// logic for FCFS disk scheduling
for(i=0;i<n;i++)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
printf("Total head moment is %d",TotalHeadMoment);
printf("\n");
return 0;
}

```

Q.1) Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance. Consider the following snapshot of system, A, B, C and D is the resource type.

Process	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0
P1	1	0	0	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	6	3	2	0	6	5	2				
P4	0	0	1	4	0	6	5	6				

a) Calculate and display the content of need matrix?

b) Is the system in safe state? If display the safe sequence.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, m;
```

```
    int allocation[10][10], max[10][10], need[10][10];
```

```
    int available[10], work[10];
```

```
    int finish[10] = {0};
```

```
    int safeSequence[10];
```

```
    int count = 0, i, j, k;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resources: ");
```

```
    scanf("%d", &m);
```

```
    /* Accept Allocation Matrix */
```

```
    printf("\nEnter Allocation Matrix:\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        for(j = 0; j < m; j++)
```

```
        {
```

```
            scanf("%d", &allocation[i][j]);
```

```
        }
```

```
    }
```

```
    /* Accept Maximum Matrix */
```

```
    printf("\nEnter Maximum Matrix:\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        for(j = 0; j < m; j++)
```

```
        {
```

```
            scanf("%d", &max[i][j]);
```

```
        }
```

```
    }
```

```

/* Calculate Need Matrix */
for(i = 0; i < n; i++)
{
    for(j = 0; j < m; j++)
    {
        need[i][j] = max[i][j] - allocation[i][j];
    }
}

/* Accept Available */
printf("\nEnter Available Resources:\n");
for(j = 0; j < m; j++)
{
    scanf("%d", &available[j]);
}

/* Display Available */
printf("\nAvailable Resources:\n");
for(j = 0; j < m; j++)
{
    printf("%d ", available[j]);
}

/* Display Need Matrix */
printf("\n\nNeed Matrix:\n");
for(i = 0; i < n; i++)
{
    for(j = 0; j < m; j++)
    {
        printf("%d ", need[i][j]);
    }
    printf("\n");
}

/* Safety Algorithm */
for(j = 0; j < m; j++)
    work[j] = available[j];

while(count < n)
{
    int found = 0;

    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int possible = 1;

```

```

        for(j = 0; j < m; j++)
        {
            if(need[i][j] > work[j])
            {
                possible = 0;
                break;
            }
        }

        if(possible)
        {
            for(k = 0; k < m; k++)
            {
                work[k] = work[k] + allocation[i][k];
            }

            safeSequence[count] = i;
            finish[i] = 1;
            count++;
            found = 1;
        }
    }
}

if(found == 0)
    break;
}

/* Check Safe State */
if(count == n)
{
    printf("\n\nSystem is in SAFE STATE.");
    printf("\nSafe Sequence: ");

    for(i = 0; i < n; i++)
    {
        printf("P%d", safeSequence[i]);

        if(i != n - 1)
            printf(" -> ");
    }
}
else
{
    printf("\n\nSystem is NOT in SAFE STATE.");
}
return 0;

```

```
}
```

Q.2) Write a simulation program for disk scheduling using SSTF algorithm.

Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.

98, 183, 37, 122, 14, 124, 65

Start Head Position: 53

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int RQ[100],i,n,TotalHeadMoment=0,initial,count=0;
    printf("Enter the number of Requests\n");
    scanf("%d",&n);
    printf("Enter the Requests sequence\n");
    for(i=0;i<n;i++)
        scanf("%d",&RQ[i]);
    printf("Enter initial head position\n");
    scanf("%d",&initial);
    // logic for sstf disk scheduling
    /* loop will execute until all process is completed*/
    while(count!=n)
    {
        int min=1000,d,index;
        for(i=0;i<n;i++)
        {
            d=abs(RQ[i]-initial);
            if(min>d)
            {
                min=d;
                index=i;
            }
        }
        TotalHeadMoment=TotalHeadMoment+min;
        initial=RQ[index];
        // 1000 is for max
        // you can use any number
        RQ[index]=1000;
        count++;
    }
    printf("Total head movement is %d",TotalHeadMoment);
    printf("\n");
    return 0;
}
```

Slip 4

Q.1) Implement the Menu driven Banker's algorithm for accepting Allocation,

Max from user

a) Accept Available

b) Display Allocation, Max

c) Find Need and display It,

d) Display Available

Consider the system with 3 resources types A, B, and C with 7, 2, 6 instances

Respectively consider the following snapshot:

```
#include <stdio.h>
int main()
{
    int n, m;
    int allocation[10][10], max[10][10], need[10][10];
    int available[10], work[10];
    int finish[10] = {0};
    int safeSequence[10];
    int count = 0, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    /* Accept Allocation Matrix */
    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }

    /* Accept Maximum Matrix */
    printf("\nEnter Maximum Matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
            scanf("%d", &max[i][j]);
        }
    }

    /* Calculate Need Matrix */
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
```

```

        need[i][j] = max[i][j] - allocation[i][j];
    }
}

/* Accept Available */
printf("\nEnter Available Resources:\n");
for(j = 0; j < m; j++)
{
    scanf("%d", &available[j]);
}

/* Display Available */
printf("\nAvailable Resources:\n");
for(j = 0; j < m; j++)
{
    printf("%d ", available[j]);
}

/* Display Need Matrix */
printf("\n\nNeed Matrix:\n");
for(i = 0; i < n; i++)
{
    for(j = 0; j < m; j++)
    {
        printf("%d ", need[i][j]);
    }
    printf("\n");
}

/* Safety Algorithm */
for(j = 0; j < m; j++)
    work[j] = available[j];

while(count < n)
{
    int found = 0;

    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int possible = 1;

            for(j = 0; j < m; j++)
            {
                if(need[i][j] > work[j])
                {
                    possible = 0;

```

```

        break;
    }
}

if(possible)
{
    for(k = 0; k < m; k++)
    {
        work[k] = work[k] + allocation[i][k];
    }

    safeSequence[count] = i;
    finish[i] = 1;
    count++;
    found = 1;
}
}
}

if(found == 0)
    break;
}

/* Check Safe State */
if(count == n)
{
    printf("\n\nSystem is in SAFE STATE.");
    printf("\nSafe Sequence: ");

    for(i = 0; i < n; i++)
    {
        printf("P%d", safeSequence[i]);

        if(i != n - 1)
            printf(" -> ");
    }
}
else
{
    printf("\n\nSystem is NOT in SAFE STATE.");
}
return 0;
}

```

Q.2) Write a simulation program for disk scheduling using SCAN algorithm. Accept total Number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.

82, 170, 43, 140, 24, 16, 190, 65

Starting Head position= 50

Direction: Left

```
#include<stdlib.h>
int main()
{
int RQ[100],i,j,n,TotalHeadMoment=0,initial,size,move;
printf("Enter the number of Requests\n");
scanf("%d",&n);
printf("Enter the Requests sequence\n");
for(i=0;i<n;i++)
scanf("%d",&RQ[i]);
printf("Enter initial head position\n");
scanf("%d",&initial);
printf("Enter total disk size\n");
scanf("%d",&size);
printf("Enter the head movement direction for high 1 and for low 0\n");
scanf("%d",&move);
// logic for Scan disk scheduling
/*logic for sort the request array */
for(i=0;i<n;i++)
{
for(j=0;j<n-i-1;j++)
{
if(RQ[j]>RQ[j+1])
{
int temp;
temp=RQ[j];
RQ[j]=RQ[j+1];
RQ[j+1]=temp;
}
}
}
int index;
for(i=0;i<n;i++)
{
if(initial<RQ[i])
{
index=i;
break;
}
}
// if movement is towards high value
if(move==1)
{
for(i=index;i<n;i++)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
```

```

}
// last movement for max size
TotalHeadMoment=TotalHeadMoment+abs(size-RQ[i-1]-1);
initial = size-1;
for(i=index-1;i>=0;i--)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
}
// if movement is towards low value
else
{
for(i=index-1;i>=0;i--)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
}
// last movement for min size
TotalHeadMoment=TotalHeadMoment+abs(RQ[i+1]-0);
initial =0;
for(i=index;i<n;i++)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
}
printf("Total head movement is %d",TotalHeadMoment);
return 0;
}

```

Slip 5

Q.1) Write the simulation program for demand paging and show the page scheduling total and number of page faults according the FIFO page replacement algorithm. Assume the memory of n frames.

Reference String: 3,4,5,6,3,4,7,3,4,5,6,7,2,4,6

```

#include<stdio.h>
#define MAX 20
int frames[MAX],ref[MAX],mem[MAX][MAX],faults,sp,m,n;
void accept()
{
int i;
printf("Enter no.of frames:");
scanf("%d", &n);
printf("Enter no.of references:");
scanf("%d", &m);
printf("Enter reference string:\n");
for(i=0;i<m;i++)
{

```

```

printf("[%d]=",i);
scanf("%d",&ref[i]);
}
}
void disp()
{
int i,j;
for(i=0;i<m;i++)
printf("%3d",ref[i]);
printf("\n\n");
for(i=0;i<n;i++)
{
for(j=0;j<m;j++)
{
if(mem[i][j])
printf("%3d",mem[i][j]);
else
printf(" ");
}
printf("\n");
}
printf("Total Page Faults: %d\n",faults);
}
int search(int pno)
{
int i;
for(i=0;i<n;i++)
{
if(frames[i]==pno)
return i;
}
return -1;
}
void fifo()
{
int i,j;
for(i=0;i<m;i++)
{
if(search(ref[i])==-1)
{
frames[sp] = ref[i];
sp = (sp+1)%n;
faults++;
for(j=0;j<n;j++)
mem[j][i] = frames[j];
}
}
}
}

```

```

int main()
{
accept();
fifo();
disp();
return 0;
}

```

Q.2) Consider a system with 'n' processes and 'm' resource types. Accept number of instances for every resource type. For each process accept the allocation and maximum requirement matrices. Write a program to display the contents of need matrix and to check if the given request of a process can be granted immediately or not. (Use resource request algorithm)

```

#include <stdio.h>
#define MAX 20
int main()
{
int n, m;
int allocation[MAX][MAX];
int maximum[MAX][MAX];
int need[MAX][MAX];
int available[MAX];
int request[MAX];
int work[MAX];
int finish[MAX];
int process;
int i, j;
int safe;
/* Accept number of processes and resources */
printf("Enter number of processes: ");
scanf("%d", &n);
printf("Enter number of resource types: ");
scanf("%d", &m);
/* Accept available instances */
printf("\nEnter available instances of each resource:\n");
for (j = 0; j < m; j++)
{
printf("Resource R%d: ", j);
scanf("%d", &available[j]);
}
/* Accept Allocation Matrix */
printf("\nEnter Allocation Matrix:\n");
for (i = 0; i < n; i++)
{
printf("Process P%d: ", i);
for (j = 0; j < m; j++)
{
scanf("%d", &allocation[i][j]);
}
}
}

```

```

    }
    /* Accept Maximum Matrix */
    printf("\nEnter Maximum Requirement Matrix:\n");
    for (i = 0; i < n; i++)
    {
        printf("Process P%d: ", i);
        for (j = 0; j < m; j++)
        {
            scanf("%d", &maximum[i][j]);
        }
    }

    /* Calculate Need Matrix */
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < m; j++)
        {
            need[i][j] = maximum[i][j] - allocation[i][j];
        }
    }
    /* Display Need Matrix */
    printf("\nNeed Matrix:\n");
    printf("    ");
    for (j = 0; j < m; j++)
        printf("R%d ", j);
    printf("\n");
    for (i = 0; i < n; i++)
    {
        printf("P%d   ", i);

        for (j = 0; j < m; j++)
        {
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }
    /* Accept process making request */
    printf("\nEnter process number making the request: ");
    scanf("%d", &process);
    /* Accept request */
    printf("Enter request for Process P%d:\n", process);
    for (j = 0; j < m; j++)
    {
        printf("R%d: ", j);
        scanf("%d", &request[j]);
    }
    /*

```

Step 1:

```

    Check Request <= Need
*/
for (j = 0; j < m; j++)
{
    if (request[j] > need[process][j])
    {
        printf("\nError: Process has requested more than its maximum need.\n");
        return 0;
    }
}
/*
Step 2:
    Check Request <= Available
*/
for (j = 0; j < m; j++)
{
    if (request[j] > available[j])
    {
        printf("\nRequest cannot be granted immediately.\n");
        printf("Reason: Required resources are not currently available.\n");
        return 0;
    }
}
/*
Step 3:
    Temporarily allocate requested resources
*/
for (j = 0; j < m; j++)
{
    available[j] -= request[j];
    allocation[process][j] += request[j];
    need[process][j] -= request[j];
}
/*
Step 4:
    Safety Algorithm
*/
for (j = 0; j < m; j++)
    work[j] = available[j];
for (i = 0; i < n; i++)
    finish[i] = 0;
safe = 1;
while (1)
{
    int found = 0;
    for (i = 0; i < n; i++)
    {
        if (finish[i] == 0)

```

```

        {
            int possible = 1;
            for (j = 0; j < m; j++)
            {
                if (need[i][j] > work[j])
                {
                    possible = 0;
                    break;
                }
            }
        }
        if (possible == 1)
        {
            /*
             * Process can complete.
             * Release its allocated resources.
             */
            for (j = 0; j < m; j++)
                work[j] += allocation[i][j];
            finish[i] = 1;
            found = 1;
        }
    }
}
if (found == 0)
    break;
}
/* Check whether all processes can finish */
for (i = 0; i < n; i++)
{
    if (finish[i] == 0)
    {
        safe = 0;
        break;
    }
}
/*
 * Step 5:
 * Grant or reject request
 */
if (safe == 1)
{
    printf("\nRequest CAN be granted immediately.\n");
    printf("System remains in a SAFE state.\n");
    printf("\nSafe State: YES\n");
}
else
{

```

```

printf("\nRequest CANNOT be granted immediately.\n");
printf("Granting the request may lead to an UNSAFE state.\n");
printf("Resources should NOT be allocated.\n");
printf("\nSafe State: NO\n");
return 0;
}

```

Slip 6

Q.1) Write a program to simulate Sequential file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned below and implement each option.

- **Show Bit Vector**
- **Create New File**
- **Show Directory**
- **Exit**

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#define MAX 200
typedef struct dir
{
char fname[20];
int start;
struct dir *next;
}NODE;
NODE *first,*last;
int n,fb,bit[MAX];
void init()
{
int i;
printf("Enter total no.of disk blocks:");
scanf("%d",&n);
fb = n;
for(i=0;i<10;i++)
{
int k = rand()%n;
if(bit[k]!=-2)
{
bit[k]=-2;

fb--;
}
}
}
void show_bitvector()
{
int i;
for(i=0;i<n;i++)

```

```

printf("%d ",bit[i]);
printf("\n");
}
void show_dir()
{
NODE *p;
int i;
printf("File\tChain\n");
p = first;
while(p!=NULL)
{
printf("%s\t",p->fname);
i = p->start;
while(i!=-1)
{
printf("%d->",i);
i=bit[i];
}
printf("NULL\n");
p=p->next;
}
}
void create()
{
NODE *p;
char fname[20];
int i,j,nob;
printf("Enter file name:");
scanf("%s",fname);
printf("Enter no.of blocks:");
scanf("%d",&nob);
if(nob>fb)
{
printf("Failed to create file %s\n",fname);
return;
}
for(i=0;i<n;i++)
{
if(bit[i]==0) break;
}
p = (NODE*)malloc(sizeof(NODE));
strcpy(p->fname,fname);
p->start=i;
p->next=NULL;
if(first==NULL)
first=p;
else
last->next=p;
}

```

```

last=p;
fb-=nob;
j=i+1;
nob--;
while(nob>0)
{
if(bit[j]==0)
{
bit[i]=j;
i=j;
nob--;
}
j++;
}
bit[i]=- 1;
printf("File %s created successully.\n",fname);
}
void delete()
{
char fname[20];
NODE *p,*q;
int nob=0,i,j;
printf("Enter file name to be deleted:");
scanf("%s",fname);
p = q = first;
while(p!=NULL)
{
if(strcmp(p->fname,fname)==0)
break;
q=p;
p=p->next;
}
if(p==NULL)
{
printf("File %s not found.\n",fname);
return;
}
i = p->start;
while(i!=-1)
{
nob++;
j = i;
i = bit[i];
bit[j] = 0;
}
fb+=nob;
if(p==first)
first=first->next;

```

```

else if(p==last)
{
last=q;
last->next=NULL;
}
else
q->next = p->next;
free(p);
printf("File %s deleted successfully.\n",fname);
}
int main()
{
int ch;
init();
while(1)
{
printf("1.Show bit vector\n");
printf("2.Create new file\n");
printf("3.Show directory\n");
printf("4.Delete file\n");
printf("5.Exit\n");
printf("Enter your choice (1-5):");
scanf("%d",&ch);
switch(ch)
{
case 1:
show_bitvector();
break;
case 2:
create();
break;
case 3:
show_dir();
break;
case 4:
delete();
break;
case 5:
exit(0);
}
}
return 0;
}

```

Q.2) Write a simulation program for disk scheduling using C-SCAN algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. also display the total number of head movements (Assume disk size=200) 15, 45, 75, 105, 135, 165, 195, 80

Starting Head Position: 100

Direction: Right

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int RQ[100],i,j,n,TotalHeadMoment=0,initial,size,move;
    printf("Enter the number of Requests\n");
    scanf("%d",&n);
    printf("Enter the Requests sequence\n");
    for(i=0;i<n;i++)
        scanf("%d",&RQ[i]);
    printf("Enter initial head position\n");
    scanf("%d",&initial);
    printf("Enter total disk size\n");
    scanf("%d",&size);
    printf("Enter the head movement direction for high 1 and for low 0\n");
    scanf("%d",&move);
    // logic for C-Scan disk scheduling
    /*logic for sort the request array */
    for(i=0;i<n;i++)
    {
        for( j=0;j<n-i-1;j++)
        {
            if(RQ[j]>RQ[j+1])
            {
                int temp;
                temp=RQ[j];
                RQ[j]=RQ[j+1];
                RQ[j+1]=temp;
            }
        }
    }
    int index;
    for(i=0;i<n;i++)
    {
        if(initial<RQ[i])
        {
            index=i;
            break;
        }
    }
    // if movement is towards high value
    if(move==1)
    {
        for(i=index;i<n;i++)
        {
            TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
        }
    }
}
```

```

        initial=RQ[i];
    }
    // last movement for max size
    TotalHeadMoment=TotalHeadMoment+abs(size-RQ[i-1]-1);
    /*movement max to min disk */
    TotalHeadMoment=TotalHeadMoment+abs(size-1-0);
    initial=0;
    for( i=0;i<index;i++)
    {
        TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
        initial=RQ[i];
    }
}
// if movement is towards low value
else
{
    for(i=index-1;i>=0;i--)
    {
        TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
        initial=RQ[i];
    }
    // last movement for min size
    TotalHeadMoment=TotalHeadMoment+abs(RQ[i+1]-0);
    /*movement min to max disk */
    TotalHeadMoment=TotalHeadMoment+abs(size-1-0);
    initial =size-1;
    for(i=n-1;i>=index;i--)
    {
        TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
        initial=RQ[i];
    }
}
printf("Total head movement is %d",TotalHeadMoment);
return 0;
}

```

Slip 7

Q.1) Consider the following snapshot of the system.

Process	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	2	0	0	1	4	2	1	2	3	3	2	1
P1	3	1	2	1	5	2	5	2				
P2	2	1	0	3	2	3	1	6				
P3	1	3	1	2	1	4	2	4				
P4	1	4	3	2	3	6	6	5				

Using Resource –Request algorithm to Check whether the current system is in safe state or not

```
#include <stdio.h>
```

```

#define MAX 20
int main()
{
    int n, m;
    int allocation[MAX][MAX];
    int maximum[MAX][MAX];
    int need[MAX][MAX];
    int available[MAX];
    int request[MAX];
    int work[MAX];
    int finish[MAX];
    int process;
    int i, j;
    int safe;
    /* Accept number of processes and resources */
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);
    /* Accept available instances */
    printf("\nEnter available instances of each resource:\n");
    for (j = 0; j < m; j++)
    {
        printf("Resource R%d: ", j);
        scanf("%d", &available[j]);
    }
    /* Accept Allocation Matrix */
    printf("\nEnter Allocation Matrix:\n");
    for (i = 0; i < n; i++)
    {
        printf("Process P%d: ", i);
        for (j = 0; j < m; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }
    /* Accept Maximum Matrix */
    printf("\nEnter Maximum Requirement Matrix:\n");
    for (i = 0; i < n; i++)
    {
        printf("Process P%d: ", i);
        for (j = 0; j < m; j++)
        {
            scanf("%d", &maximum[i][j]);
        }
    }

    /* Calculate Need Matrix */

```

```

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < m; j++)
        {
            need[i][j] = maximum[i][j] - allocation[i][j];
        }
    }
/* Display Need Matrix */
printf("\nNeed Matrix:\n");
printf("    ");
for (j = 0; j < m; j++)
    printf("R%d ", j);
printf("\n");
for (i = 0; i < n; i++)
{
    printf("P%d   ", i);

    for (j = 0; j < m; j++)
    {
        printf("%d ", need[i][j]);
    }
    printf("\n");
}
/* Accept process making request */
printf("\nEnter process number making the request: ");
scanf("%d", &process);
/* Accept request */
printf("Enter request for Process P%d:\n", process);
for (j = 0; j < m; j++)
{
    printf("R%d: ", j);
    scanf("%d", &request[j]);
}
/*
    Step 1:
    Check Request <= Need
*/
for (j = 0; j < m; j++)
{
    if (request[j] > need[process][j])
    {
        printf("\nError: Process has requested more than its maximum need.\n");
        return 0;
    }
}
/*
    Step 2:
    Check Request <= Available

```

```

    */
    for (j = 0; j < m; j++)
    {
        if (request[j] > available[j])
        {
            printf("\nRequest cannot be granted immediately.\n");
            printf("Reason: Required resources are not currently available.\n");
            return 0;
        }
    }
}
/*
    Step 3:
    Temporarily allocate requested resources
    */
    for (j = 0; j < m; j++)
    {
        available[j] -= request[j];
        allocation[process][j] += request[j];
        need[process][j] -= request[j];
    }
/*
    Step 4:
    Safety Algorithm
    */
    for (j = 0; j < m; j++)
        work[j] = available[j];
    for (i = 0; i < n; i++)
        finish[i] = 0;
    safe = 1;
    while (1)
    {
        int found = 0;
        for (i = 0; i < n; i++)
        {
            if (finish[i] == 0)
            {
                int possible = 1;
                for (j = 0; j < m; j++)
                {
                    if (need[i][j] > work[j])
                    {
                        possible = 0;
                        break;
                    }
                }
            }
        }
        if (possible == 1)
        {
            /*

```

```

        Process can complete.
        Release its allocated resources.
    */
    for (j = 0; j < m; j++)
        work[j] += allocation[i][j];
    finish[i] = 1;
    found = 1;
}
}
}
if (found == 0)
    break;
}
/* Check whether all processes can finish */
for (i = 0; i < n; i++)
{
    if (finish[i] == 0)
    {
        safe = 0;
        break;
    }
}
/*
    Step 5:
    Grant or reject request
*/
if (safe == 1)
{
    printf("\nRequest CAN be granted immediately.\n");
    printf("System remains in a SAFE state.\n");
    printf("\nSafe State: YES\n");
}
else
{
}
printf("\nRequest CANNOT be granted immediately.\n");
printf("Granting the request may lead to an UNSAFE state.\n");
printf("Resources should NOT be allocated.\n");
printf("\nSafe State: NO\n");
return 0;
}

```

Q2) Write a program that demonstrates the use of nice() system call. After a child process is started using fork(), assign higher priority to the child using nice() system call.

```

#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()

```

```

{
pid_t pid;
pid = fork();
if (pid == 0)
{
printf("\nI am child process, id=%d\n",getpid());
printf("\nPriority :%d,id=%d\n",nice (-7),getpid());
}
else
{
printf("\nI am parent process, id=%d\n",getpid());
nice(1);
printf("\nPriority :%d,id=%d\n",nice (15),getpid());
}
return 0;
}

```

Slip 8

Q.1) Write a program to simulate Index file allocation method. Assume disk with 'n' number of blocks. Give value of 'n' as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driven program with menu options as mentioned above and implement each option

- **Show Bit Vector**
- **Create New File**
- **Show Directory**
- **Exit**

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#define MAX 200
typedef struct dir
{
char fname[20];
int start,length;
struct dir *next;
}NODE;
NODE *first,*last;
int bit[MAX],n;
void init()
{
int i;
printf("Enter total no.of disk blocks:");
scanf("%d",&n);
for(i=0;i<n;i++)
{
bit[i] =rand()%2;
}
}
void show_bitvector()

```

```

{
int i;
for(i=0;i<n;i++)
{
printf("%d",bit[i]);
}
printf("\n");
}
void show_dir()
{
NODE *p;
printf("File\tIndexed block\n");
p = first;
while(p!=NULL)
{
printf("%s\t%d\n",
p->fname,p->length);
p = p->next;
}
}
void create()
{
NODE *p;
char fname[20];
int nob,i=0,j=0,start;
printf("Enter file name:");
scanf("%s",fname);
printf("Enter no.of blocks:");
scanf("%d",&nob);
while(1)
{
while(i<n)
{
if(bit[i]==0) break;
i++;
}
if(i<n)
{
start = i;
j=1;
while(j<nob && i<n && bit[i]==0)
{
i++;
j++;
}
if(j==nob)
{
p = (NODE*)malloc(sizeof(NODE));

```

```

strcpy(p->fname,fname);
p->start = start;
p->length = nob;
p->next = NULL;
if(first==NULL)
first=p;
else
last->next=p;
last=p;
for(j=0;j<nob;j++)
bit[j+start]=1;
printf("File %s created successfully.\n",fname);
return;
}
}
else
{
printf("Fail to create file %s\n",fname);
return;
}
}
}
void delete()
{
NODE *p,*q;
char fname[20];
int i;
printf("Enter file to be deleted:");
scanf("%s",fname);
p = q = first;
while(p!=NULL)
{
if(strcmp(p->fname,fname)==0)
break;
q = p;
p = p->next;
}
if(p==NULL)
{
printf("File %s not found.\n",fname);
return;
}
for(i=0;i<p->length;i++)
bit[p->start+i]=0;
if(p==first)
first = first->next;
else if(p==last)
{

```

```

last=q;
last->next=NULL;
}
else
{
q->next = p->next;
}
free(p);
printf("File %s deleted successfully.\n",fname);
}
void main()
{
int ch;
init();
while(1)
{
printf("1.Show bit vector\n");
printf("2.Create new file\n");
printf("3.Show directory\n");
printf("4.Delete file\n");
printf("5.Exit\n");
printf("Enter your choice (1-5):");
scanf("%d",&ch);
switch(ch)
{
case 1:
show_bitvector();
break;
case 2:
create();
break;
case 3:
show_dir();
break;
case 4:
delete();
break;
case 5:
exit(0);
}
}
}

```

Q.2) Write a program that demonstrates the use of nice () system call. After a child process is started using fork(), assign higher priority to the child using nice() system call

```

#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

```

```

int main()
{
    pid_t pid;
    pid = fork();
    if (pid == 0)
    {
        printf("\nI am child process, id=%d\n",getpid());
        printf("\nPriority :%d,id=%d\n",nice (-7),getpid());
    }
    else
    {
        printf("\nI am parent process, id=%d\n",getpid());
        nice(1);
        printf("\nPriority :%d,id=%d\n",nice (15),getpid());
    }
    return 0;
}

```

Slip 9

Q.1) Write the simulation program for demand paging and show the page scheduling and total number of page faults according the optimal page replacement algorithm. Assume the memory of 'n' frames.

Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2

```
#include<stdio.h>
```

```

int main()
{
    int no_of_frames, no_of_pages, frames[10], pages[30], temp[10], flag1, flag2, flag3, i, j, k,
    pos,
    max, faults = 0;
    printf("Enter number of frames: ");
    scanf("%d", &no_of_frames);

    printf("Enter number of pages: ");
    scanf("%d", &no_of_pages);

    printf("Enter page reference string: ");

    for(i = 0; i < no_of_pages; ++i){
        scanf("%d", &pages[i]);
    }

    for(i = 0; i < no_of_frames; ++i){
        frames[i] = -1;
    }

    for(i = 0; i < no_of_pages; ++i){
        flag1 = flag2 = 0;

        for(j = 0; j < no_of_frames; ++j){

```

```

        if(frames[j] == pages[i]){
            flag1 = flag2 = 1;
            break;
        }
    }

    if(flag1 == 0){
        for(j = 0; j < no_of_frames; ++j){
            if(frames[j] == -1){
                faults++;
                frames[j] = pages[i];
                flag2 = 1;
                break;
            }
        }
    }
}

if(flag2 == 0){
    flag3 = 0;

    for(j = 0; j < no_of_frames; ++j){
        temp[j] = -1;

        for(k = i + 1; k < no_of_pages; ++k){
            if(frames[j] == pages[k]){
                temp[j] = k;
                break;
            }
        }
    }

    for(j = 0; j < no_of_frames; ++j){
        if(temp[j] == -1){
            pos = j;
            flag3 = 1;
            break;
        }
    }

    if(flag3 == 0){
        max = temp[0];
        pos = 0;

        for(j = 1; j < no_of_frames; ++j){
            if(temp[j] > max){
                max = temp[j];
                pos = j;
            }
        }
    }
}

```

```

        }
    }
    frames[pos] = pages[i];
    faults++;
}

printf("\n");

for(j = 0; j < no_of_frames; ++j){
    printf("%d\t", frames[j]);
}
}

printf("\n\nTotal Page Faults = %d", faults);

return 0;
}

```

Q.2) Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. the arrival time and first CPU burst for different n number of processes should be input to the algorithm. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
typedef struct process_info
{
    char pname[20];
    int at, bt, ct, bt1;
    struct process_info *next;
}NODE;
int n;
NODE *first, *last;
void accept_info()
{
    NODE *p;
    int i;
    printf("Enter no.of process:");
    scanf("%d", &n);
    for(i=0; i<n; i++)
    {
        p = (NODE*)malloc(sizeof(NODE));
        printf("Enter process name:");
        scanf("%s", p->pname);
        printf("Enter arrival time:");
        scanf("%d", &p->at);
        printf("Enter first CPU burst time:");
        scanf("%d", &p->bt);
    }
}

```

```

p->bt1 = p->bt;
p->next = NULL;
if(first==NULL)
first=p;
else
last->next=p;
last = p;
}
}
void print_output()
{
NODE *p;
float avg_tat=0,avg_wt=0;
printf("pname\tat\tbt\tct\ttat\twt\n");
p = first;
while(p!=NULL)
{
Int tat = p->ct-p->at;
int wt = tat-p->bt;
avg_tat+=tat;
avg_wt+=wt;
printf("%s\t%d\t%d\t%d\t%d\t%d\n",
p->pname,p->at,p->bt,p->ct,tat,wt);
p=p->next;
}
printf("Avg TAT=%f\tAvg WT=%f\n",
avg_tat/n,avg_wt/n);
}
void print_input()
{
NODE *p;
p = first;
printf("pname\tat\tbt\n");
while(p!=NULL)
{
printf("%s\t%d\t%d\n",
p->pname,p->at,p->bt1);
p = p->next;
}
}
void sort()
{
NODE *p,*q;
int t;
char name[20];
p = first;
while(p->next!=NULL)
{

```

```

q=p->next;
while(q!=NULL)
{
if(p->at > q->at)
{
strcpy(name,p->pname);
strcpy(p->pname,q->pname);
strcpy(q->pname,name);
t = p->at;
p->at = q->at;
q->at = t;
t = p->bt;
p->bt = q->bt;
q->bt = t;
t = p->ct;
p->ct = q->ct;
q->ct = t;
t = p->bt1;
p->bt1 = q->bt1;
q->bt1 = t;
}
q=q->next;
}
p=p->next;
}
int time;
NODE * get_sjf()
{
NODE *p,*min_p=NULL;
int min=9999;
p = first;
while(p!=NULL)
{
if(p->at<=time && p->bt1!=0 &&
p->bt1<min)
{
min = p->bt1;
min_p = p;
}
p=p->next;
}
return min_p;
}
struct gantt_chart
{
int start;
char pname[30];

```

```

int end;
}s[100],s1[100];
int k;
void sjfnp()
{
int prev=0,n1=0;
NODE *p;
while(n1!=n)
{
p = get_sjf();
if(p==NULL)
{
time++;
s[k].start = prev;
strcpy(s[k].pname,"");
s[k].end = time;
prev = time;
k++;
}
else
{
time+=p->bt1;
s[k].start = prev;
strcpy(s[k].pname, p->pname);
s[k].end = time;
prev = time;
k++;
p->ct = time;
p->bt1 = 0;
n1++;
}
print_input();
sort();
}
}
void print_gantt_chart()
{
int i,j,m;
s1[0] = s[0];
for(i=1,j=0;i<k;i++)
{
if(strcmp(s[i].pname,s1[j].pname)==0)
s1[j].end = s[i].end;
else
s1[++j] = s[i];
}
printf("%d",s1[0].start);
for(i=0;i<=j;i++)

```

```

{
m = (s1[i].end - s1[i].start);
for(k=0;k<m/2;k++)
printf("-");
printf("%s",s1[i].pname);
for(k=0;k<(m+1)/2;k++)
printf("-");
printf("%d",s1[i].end);
}
}
int main()
{
accept_info();
sort();
sjfnp();
print_output();
print_gantt_chart();
return 0;
}

```

Slip 10

Q.1) Write the program to simulate Round Robin (RR) scheduling. The arrival time and first CPU burst for different n number of processes should be input to the algorithm. Also give the time quantum as input. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
typedef struct process_info
{
char pname[20];
int at, bt, ct, bt1;
struct process_info *next;
}NODE;
int n, ts;
NODE *first, *last;
void accept_info()
{
NODE *p;
int i;
printf("Enter no.of process:");
scanf("%d",&n);
for(i=0;i<n;i++)
{
p = (NODE*)malloc(sizeof(NODE));
printf("Enter process name:");
scanf("%s",p->pname);

```

```

printf("Enter arrival time:");
scanf("%d",&p->at);
printf("Enter first CPU burst time:");
scanf("%d",&p->bt);
p->bt1 = p->bt;
p->next = NULL;
if(first==NULL)
first=p;
else
last->next=p;
last = p;
}
printf("Enter time slice:");
scanf("%d",&ts);
}
void print_output()
{
NODE *p;
float avg_tat=0,avg_wt=0;
printf("pname\tat\tbt\tct\ttat\twt\n");
p = first;
while(p!=NULL)
{
int tat = p->ct-p->at;
int wt = tat-p->bt;
avg_tat+=tat;
avg_wt+=wt;
printf("%s\t%d\t%d\t%d\t%d\t%d\n",
p->pname,p->at,p->bt,p->ct,tat,wt);
p=p->next;
}
printf("Avg TAT=%f\tAvg WT=%f\n",
avg_tat/n,avg_wt/n);
}
void print_input()
{
NODE *p;
p = first;
printf("pname\tat\tbt\n");
while(p!=NULL)
{
printf("%s\t%d\t%d\n",
p->pname,p->at,p->bt1);
p = p->next;
}
}
void sort()
{

```

```

NODE *p,*q;
int t;
char name[20];
p = first;
while(p->next!=NULL)
{
q=p->next;
while(q!=NULL)
{
if(p->at > q->at)
{
strcpy(name,p->pname);
strcpy(p->pname,q->pname);
strcpy(q->pname,name);
t = p->at;
p->at = q->at;
q->at = t;
t = p->bt;
p->bt = q->bt;
q->bt = t;
t = p->ct;
p->ct = q->ct;
q->ct = t;
t = p->bt1;
p->bt1 = q->bt1;
q->bt1 = t;
}
q=q->next;
}
p=p->next;
}
int time;
int is_arrived()
{
NODE *p;
p = first;
while(p!=NULL)
{
if(p->at<=time && p->bt1!=0)
return 1;
p=p->next;
}
return 0;
}
NODE * delq()
{
NODE *t;

```

```

t = first;
first = first->next;
t->next=NULL;
return t;
}
void addq(NODE *t)
{
last->next = t;
last = t;
}
struct gantt_chart
{
int start;
char pname[30];
int end;
}s[100],s1[100];
int k;
void rr()
{
int prev=0,n1=0;
NODE *p;
while(n1!=n)
{
if(!is_arrived())
{
time++;
s[k].start = prev;
strcpy(s[k].pname,"*");
s[k].end = time;
k++;
prev=time;
}
else
{
p = first;
while(1)
{
if(p->at<=time && p->bt1!=0)
break;
p = delq();
addq(p);
p = first;
}
if(p->bt1<=ts)
{
time+=p->bt1;
p->bt1=0;
}
}
}

```

```

else
{
time+=ts;
p->bt1-=ts;
}
p->ct = time;
s[k].start = prev;
strcpy(s[k].pname,p->pname);
s[k].end = time;
k++;
prev = time;
if(p->bt1==0) n1++;
p = delq();
addq(p);
}
print_input();
}
}
void print_gantt_chart()
{
int i,j,m;
s1[0] = s[0];
for(i=1,j=0;i<k;i++)
{
if(strcmp(s[i].pname,s1[j].pname)==0)
s1[j].end = s[i].end;
else
s1[++j] = s[i];
}
printf("%d",s1[0].start);
for(i=0;i<=j;i++)
{
m = (s1[i].end - s1[i].start);
for(k=0;k<m/2;k++)
printf("-");
printf("%s",s1[i].pname);
for(k=0;k<(m+1)/2;k++)
printf("-");
printf("%d",s1[i].end);
}
}
int main()
{
accept_info();
sort();
rr();
print_output();
print_gantt_chart();
}

```

```
return 0;
}
```

Q.2) Write a simulation program for disk scheduling using Look algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of request in the order in which it is served. Also display the total number of head moments.

55, 58, 39, 18, 90, 160, 150, 38.

Start Head Position: 100

Direction: Left

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int RQ[100],i,j,n,TotalHeadMoment=0,initial,size,move;
    printf("Enter the number of Requests\n");
    scanf("%d",&n);
    printf("Enter the Requests sequence\n");
    for(i=0;i<n;i++)
        scanf("%d",&RQ[i]);
    printf("Enter initial head position\n");
    scanf("%d",&initial);
    printf("Enter total disk size\n");
    scanf("%d",&size);
    printf("Enter the head movement direction for high 1 and for low 0\n");
    scanf("%d",&move);
    // logic for look disk scheduling
    /*logic for sort the request array */
    for(i=0;i<n;i++)
    {
        for(j=0;j<n-i-1;j++)
        {
            if(RQ[j]>RQ[j+1])
            {
                int temp;
                temp=RQ[j];
                RQ[j]=RQ[j+1];
                RQ[j+1]=temp;
            }
        }
    }
    int index;
    for(i=0;i<n;i++)
    {
        if(initial<RQ[i])
        {
            index=i;
            break;
        }
    }
}
```

```

}
}
// if movement is towards high value
if(move==1)
{
for(i=index;i<n;i++)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
for(i=index-1;i>=0;i--)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
}
// if movement is towards low value
else
{
for(i=index-1;i>=0;i--)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
for(i=index;i<n;i++)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
}
printf("Total head movement is %d",TotalHeadMoment);
return 0;
}

```

Slip 11

Q.1 Write a C program to simulate Banker's algorithm for the purpose of Deadlock avoidance. The following snapshot of system, A, B, C and D are the resource type.

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	0	0	0	0	0	0
P1	2	0	0	2	0	2			
P2	3	0	3	0	0	0			
P3	2	1	1	1	0	0			
P4	0	0	2	0	0	2			

Implement the following Menu.

- a. Accept Available**
- b. Display Allocation, Max**
- c. Display the contents of need matrix**
- d. Display Available**

```
#include<stdio.h>
#define MAX 10
int m,n,avail[MAX],alloc[MAX][MAX],
max[MAX][MAX],need[MAX][MAX];
void accept()
{
    int i,j;
    printf("Enter no.of process:");
    scanf("%d",&n);
    printf("Enter no.of resource types:");
    scanf("%d",&m);
    printf("Enter no.of allocated resources of each resource type by each process:\n");
    for(i=0;i<n;i++)
    {
        printf("P%d:\n",i);
        for(j=0;j<m;j++)
        {
            printf("%c:",65+j);
            scanf("%d",&alloc[i][j]);
        }
    }
    printf("Enter no.of maximum resources of each resource type by each process:\n");
    for(i=0;i<n;i++)
    {
        printf("P%d:\n",i);
        for(j=0;j<m;j++)
        {
            printf("%c:",65+j);
            scanf("%d",&max[i][j]);
        }
    }
}
void calc_need()
{
    int i,j;
    for(i=0;i<n;i++)
    for(j=0;j<m;j++)
        need[i][j]=max[i][j]-alloc[i][j];
}
void printt()
{
    int i,j;
    printf("\tAllocation\tMax\tNeed\n\t");
    for(i=0;i<3;i++)
```

```

{
for(j=0;j<m;j++)
printf("%3c",65+j);
printf("\t");
}
printf("\n");
for(i=0;i<n;i++)
{
printf("P%d\t",i);
for(j=0;j<m;j++)
printf("%3d",alloc[i][j]);
printf("\t");
for(j=0;j<m;j++)
printf("%3d",max[i][j]);
printf("\t");
for(j=0;j<m;j++)
printf("%3d",need[i][j]);
printf("\n");
}
}
int main()
{
int i,j,pno;
accept();
int choice;
do{
printf("\n Press 1 Enter availble matrix \n Press 2 to Display allocation and max ,need \n Press 3
to
Display Avilable \n press 4 to Exit ");
printf("\n Enter your choice:");
scanf("%d", &choice);
switch (choice)
{
case 1:
printf("Enter Available:\n");
for(i=0;i<m;i++)
{
scanf("%d",&avail[i]);
}
break;
case 2:
calc_need();
printt();
break;
case 3:
printf("Available\n");
for(j=0;j<m;j++)
printf("%3d",avail[j]);

```

```

        printf("\t");
        printf("\n");
        break;
    exit(1);
    default:
        printf("\nInvalid choice\n");
        break;
    }
}
while(choice!=0);
return 0;
}

```

Q.2) Write the simulation program for demand paging and show the page scheduling and total number of page faults according the optimal page replacement algorithm. Assume the memory of n frames.

Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2

```

#include<stdio.h>
int main()
{
    int no_of_frames, no_of_pages, frames[10], pages[30], temp[10], flag1, flag2, flag3, i, j, k,
    pos,
    max, faults = 0;
    printf("Enter number of frames: ");
    scanf("%d", &no_of_frames);

    printf("Enter number of pages: ");
    scanf("%d", &no_of_pages);

    printf("Enter page reference string: ");

    for(i = 0; i < no_of_pages; ++i){
        scanf("%d", &pages[i]);
    }

    for(i = 0; i < no_of_frames; ++i){
        frames[i] = -1;
    }

    for(i = 0; i < no_of_pages; ++i){
        flag1 = flag2 = 0;

        for(j = 0; j < no_of_frames; ++j){
            if(frames[j] == pages[i]){
                flag1 = flag2 = 1;
                break;
            }
        }
    }
}

```

```

if(flag1 == 0){
    for(j = 0; j < no_of_frames; ++j){
        if(frames[j] == -1){
            faults++;
            frames[j] = pages[i];
            flag2 = 1;
            break;
        }
    }
}

if(flag2 == 0){
    flag3 = 0;

    for(j = 0; j < no_of_frames; ++j){
        temp[j] = -1;

        for(k = i + 1; k < no_of_pages; ++k){
            if(frames[j] == pages[k]){
                temp[j] = k;
                break;
            }
        }
    }

    for(j = 0; j < no_of_frames; ++j){
        if(temp[j] == -1){
            pos = j;
            flag3 = 1;
            break;
        }
    }

    if(flag3 == 0){
        max = temp[0];
        pos = 0;

        for(j = 1; j < no_of_frames; ++j){
            if(temp[j] > max){
                max = temp[j];
                pos = j;
            }
        }
    }
    frames[pos] = pages[i];
    faults++;
}

```

```

        printf("\n");

        for(j = 0; j < no_of_frames; ++j){
            printf("%d\t", frames[j]);
        }
    }

    printf("\n\nTotal Page Faults = %d", faults);

    return 0;
}

```

Slip 12

Q.1) Write the program to simulate FCFS CPU scheduling. The arrival time and first CPU- burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
typedef struct process_info
{
    char pname[20];
    int at, bt, ct, bt1;
    struct process_info *next;
}NODE;
int n;
NODE *first, *last;
void accept_info()
{
    NODE *p;
    int i;
    printf("Enter no.of process:");
    scanf("%d", &n);
    for(i=0; i<n; i++)
    {
        p = (NODE*)malloc(sizeof(NODE));
        printf("Enter process name:");
        scanf("%s", p->pname);
        printf("Enter arrival time:");
        scanf("%d", &p->at);
        printf("Enter first CPU burst time:");
        scanf("%d", &p->bt);
        p->bt1 = p->bt;
        p->next = NULL;
        if(first==NULL)
            first=p;
        else

```

```

last->next=p;
last = p;
}
}
void print_output()
{
NODE *p;
float avg_tat=0,avg_wt=0;
printf("pname\tat\tbt\tct\ttat\twt\n");
p = first;
while(p!=NULL)
{
int tat = p->ct-p->at;
int wt = tat-p->bt;
avg_tat+=tat;
avg_wt+=wt;
printf("%s\t%d\t%d\t%d\t%d\t%d\n",
p->pname,p->at,p->bt,p->ct,tat,wt);
p=p->next;
}
printf("Avg TAT=%f\tAvg WT=%f\n",
avg_tat/n,avg_wt/n);
}
void print_input()
{
NODE *p;
p = first;
printf("pname\tat\tbt\n");
while(p!=NULL)
{
printf("%s\t%d\t%d\n",
p->pname,p->at,p->bt1);
p = p->next;
}
}
void sort()
{
NODE *p,*q;
int t;
char name[20];
p = first;
while(p->next!=NULL)
{
q=p->next;
while(q!=NULL)
{
if(p->at > q->at)
{

```

```

strcpy(name,p->pname);
strcpy(p->pname,q->pname);
strcpy(q->pname,name);
t = p->at;
p->at = q->at;
q->at = t;
t = p->bt;
p->bt = q->bt;
q->bt = t;
t = p->ct;
p->ct = q->ct;
q->ct = t;
t = p->bt1;
p->bt1 = q->bt1;
q->bt1 = t;
}
q=q->next;
}
p=p->next;
}
}
int time;
NODE * get_fcfs()
{
NODE *p;
p = first;
while(p!=NULL)
{
if(p->at<=time && p->bt1!=0)
return p;
p=p->next;
}
return NULL;
}
struct gantt_chart
{
int start;
char pname[30];
int end;
}s[100],s1[100];
int k;
void fcfs()
{
int prev=0,n1=0;
NODE *p;
while(n1!=n)
{
p = get_fcfs();

```

```

if(p==NULL)
{
time++;
s[k].start = prev;
strcpy(s[k].pname, "");
s[k].end = time;
prev = time;
k++;
}
else
{
time+=p->bt1;
s[k].start = prev;
strcpy(s[k].pname, p->pname);
s[k].end = time;
prev = time;
k++;
p->ct = time;
p->bt1 = 0;
n1++;
}
print_input();
sort();
}
}
void print_gantt_chart()
{
int i,j,m;
s1[0] = s[0];
for(i=1,j=0;i<k;i++)
{
if(strcmp(s[i].pname,s1[j].pname)==0)
s1[j].end = s[i].end;
else
s1[++j] = s[i];
}
printf("%d",s1[0].start);
for(i=0;i<=j;i++)
{
m = (s1[i].end - s1[i].start);
for(k=0;k<m/2;k++)
printf("-");
printf("%s",s1[i].pname);
for(k=0;k<(m+1)/2;k++)
printf("-");
printf("%d",s1[i].end);
}
}
}

```

```

int main()
{
accept_info();
sort();
fcfs();
print_output();
print_gantt_chart();
return 0;
}

```

Q.2) Implement the C program to accept n integers to be sorted. Main function creates child process using fork system call. Parent process sorts the integers using bubble sort and waits for child process using wait system call child process sorts the integers using insertion sort

```

#include<stdio.h>
#include<sys/types.h>
#include<unistd.h>
#include<stdlib.h>
void bubblesort(int arr[30],int n)
{
int i,j,temp;
for(i=0;i<n;i++)
{
for(j=0;j<n-1;j++)
{
if(arr[j]>arr[j+1])
{
temp=arr[j];
arr[j]=arr[j+1];
arr[j+1]=temp;
}
}
}
}
void insertionsort(int arr[30], int n)
{
int i, j, temp;
for (i = 1; i < n; i++) {
temp = arr[i];
j = i - 1;

while(j>=0 && temp <= arr[j])
{
arr[j+1] = arr[j];
j = j-1;
}
arr[j+1] = temp;
}
}

```

```

void fork1()
{
    int arr[30],n,i,status;
    printf("\nEnter the no of values in array :");
    scanf("%d",&n);
    printf("\nEnter the array elements :");
    for(i=0;i<n;i++)
        scanf("%d",&arr[i]);
    int pid=fork();
    if(pid==0)
    {
        sleep(10);
        printf("\nchild process\n");
        printf("child process id=%d\n",getpid());
        insertionsort(arr,n);
        printf("\nElements Sorted Using insertion sort:");
        printf("\n");
        for(i=0;i<n;i++)
            printf("%d",arr[i]);
        printf("\b");
        printf("\nparent process id=%d\n",getppid());
        system("ps -x");
    }
    else
    {
        printf("\nparent process\n");
        printf("\nparent process id=%d\n",getppid());
        bubblesort(arr,n);
        printf("Elements Sorted Using bubble sort:");
        printf("\n");
        for(i=0;i<n;i++)
            printf("%d",arr[i]);
        printf("\n\n");
    }
}

int main()
{
    fork1();
    return 0;
}

```

Slip 13

Q.1) Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time

```

#include<stdio.h>
#include<stdlib.h>

```

```

#include<string.h>
typedef struct process_info
{
    char pname[20];
    int at, bt, ct, bt1;
    struct process_info *next;
}NODE;
int n;
NODE *first, *last;
void accept_info()
{
    NODE *p;
    int i;
    printf("Enter no. of process:");
    scanf("%d", &n);
    for(i=0; i<n; i++)
    {
        p = (NODE*)malloc(sizeof(NODE));
        printf("Enter process name:");
        scanf("%s", p->pname);
        printf("Enter arrival time:");
        scanf("%d", &p->at);
        printf("Enter first CPU burst time:");
        scanf("%d", &p->bt);
        p->bt1 = p->bt;
        p->next = NULL;
        if(first==NULL)
            first=p;
        else
            last->next=p;
        last = p;
    }
}
void print_output()
{
    NODE *p;
    float avg_tat=0, avg_wt=0;
    printf("pname\tat\tbt\tct\ttat\twt\n");
    p = first;
    while(p!=NULL)
    {
        int tat = p->ct - p->at;
        int wt = tat - p->bt;
        avg_tat += tat;
        avg_wt += wt;
        printf("%s\t%d\t%d\t%d\t%d\t%d\n",
            p->pname, p->at, p->bt, p->ct, tat, wt);
        p = p->next;
    }
}

```

```

}
printf("Avg TAT=%f\tAvg WT=%f\n",
avg_tat/n,avg_wt/n);
}
void print_input()
{
NODE *p;
p = first;
printf("pname\tat\tbt\n");
while(p!=NULL)
{
printf("%s\t%d\t%d\n",
p->pname,p->at,p->bt1);
p = p->next;
}
}
void sort()
{
NODE *p,*q;
int t;
char name[20];
p = first;
while(p->next!=NULL)
{
q=p->next;
while(q!=NULL)
{
if(p->at > q->at)
{
strcpy(name,p->pname);
strcpy(p->pname,q->pname);
strcpy(q->pname,name);
t = p->at;
p->at = q->at;
q->at = t;
t = p->bt;
p->bt = q->bt;
q->bt = t;
t = p->ct;
p->ct = q->ct;
q->ct = t;
t = p->bt1;
p->bt1 = q->bt1;
q->bt1 = t;
}
q=q->next;
}
p=p->next;
}

```

```

}
}
int time;
NODE * get_sjf()
{
NODE *p,*min_p=NULL;
int min=9999;
p = first;
while(p!=NULL)
{
if(p->at<=time && p->bt1!=0 &&
p->bt1<min)
{
min = p->bt1;
min_p = p;
}
p=p->next;
}
return min_p;
}
struct gantt_chart
{
int start;
char pname[30];
int end;
}s[100],s1[100];
int k;
void sjfnp()
{
int prev=0,n1=0;
NODE *p;
while(n1!=n)
{
p = get_sjf();
if(p==NULL)
{
time++;
s[k].start = prev;
strcpy(s[k].pname, "");
s[k].end = time;
prev = time;
k++;
}
else
{
time+=p->bt1;
s[k].start = prev;
strcpy(s[k].pname, p->pname);

```

```

s[k].end = time;
prev = time;
k++;
p->ct = time;
p->bt1 = 0;
n1++;
}
print_input();
sort();
}
}
void print_gantt_chart()
{
int i,j,m;
s1[0] = s[0];
for(i=1,j=0;i<k;i++)
{
if(strcmp(s[i].pname,s1[j].pname)==0)
s1[j].end = s[i].end;
else
s1[++j] = s[i];
}
printf("%d",s1[0].start);
for(i=0;i<=j;i++)
{
m = (s1[i].end - s1[i].start);
for(k=0;k<m/2;k++)
printf("-");
printf("%s",s1[i].pname);
for(k=0;k<(m+1)/2;k++)
printf("-");
printf("%d",s1[i].end);
}
}
int main()
{
accept_info();
sort();
sjfnp();
print_output();
print_gantt_chart();
return 0;
}

```

Q.2) Write a simulation program for disk scheduling using LOOK algorithm.

Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served.

Also display the total number of head moments

15, 30, 55, 90, 125, 140, 170

Starting Head Position: 100**Direction: Left**

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int RQ[100],i,j,n,TotalHeadMoment=0,initial,size,move;
    printf("Enter the number of Requests\n");
    scanf("%d",&n);
    printf("Enter the Requests sequence\n");
    for(i=0;i<n;i++)
        scanf("%d",&RQ[i]);
    printf("Enter initial head position\n");
    scanf("%d",&initial);
    printf("Enter total disk size\n");
    scanf("%d",&size);
    printf("Enter the head movement direction for high 1 and for low 0\n");
    scanf("%d",&move);
    // logic for look disk scheduling
    /*logic for sort the request array */
    for(i=0;i<n;i++)
    {
        for(j=0;j<n-i-1;j++)
        {
            if(RQ[j]>RQ[j+1])
            {
                int temp;
                temp=RQ[j];
                RQ[j]=RQ[j+1];
                RQ[j+1]=temp;
            }
        }
    }
    int index;
    for(i=0;i<n;i++)
    {
        if(initial<RQ[i])
        {
            index=i;
            break;
        }
    }
    // if movement is towards high value
    if(move==1)
    {
        for(i=index;i<n;i++)
        {
            TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
```

```

initial=RQ[i];
}
for(i=index-1;i>=0;i--)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
}
// if movement is towards low value
else
{
for(i=index-1;i>=0;i--)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
}
for(i=index;i<n;i++)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
}
printf("Total head movement is %d",TotalHeadMoment);
return 0;
}

```

Slip 14

Q.1) Implement the C program that accepts an integer array. Main function forks child process. Parent process sorts an integer array and passes the sorted array to child process through the command line arguments of `execve()` system call. The child process uses `execve()` system call to load new program that uses this sorted array for performing the binary search the particular item in the array.

```

//parent.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
void bubbleSort(int arr[], int n)
{
    int i, j, temp;
    for(i = 0; i < n - 1; i++)
    {
        for(j = 0; j < n - i - 1; j++)
        {
            if(arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

```

```

        }
    }
}
int main()
{
    int n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter array elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    bubbleSort(arr, n);

    pid_t pid = fork();

    if(pid > 0)
    {
        wait(NULL);

        char **args = (char **)malloc((n + 2) * sizeof(char *));

        args[0] = "./child";

        for(i = 0; i < n; i++)
        {
            args[i + 1] = (char *)malloc(20);
            sprintf(args[i + 1], "%d", arr[i]);
        }

        args[n + 1] = NULL;

        execve("./child", args, NULL);

        perror("execve failed");
    }
    else if(pid == 0)
    {
        exit(0);
    }

    return 0;
}
//child.c

```

```

#include <stdio.h>
#include <stdlib.h>

int binarySearch(int arr[], int n, int key)
{
    int low = 0, high = n - 1;

    while(low <= high)
    {
        int mid = (low + high) / 2;

        if(arr[mid] == key)
            return mid;
        else if(arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}

int main(int argc, char *argv[])
{
    int n = argc - 1;
    int arr[n];
    for(int i = 1; i < argc; i++)
        arr[i - 1] = atoi(argv[i]);
    int key;
    printf("Sorted Array: ");
    for(int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\nEnter element to search: ");
    scanf("%d", &key);
    int pos = binarySearch(arr, n, key);
    if(pos != -1)
        printf("Element found at position %d\n", pos + 1);
    else
        printf("Element not found\n");
    return 0;
}

```

Q.2) Write the simulation program for demand paging and show the page scheduling and total number of page faults according the FIFO page replacement algorithm. Assume the memory of n frames.

Reference String: 3, 4, 5, 6, 3, 4, 7, 3, 4, 5, 6, 7, 2, 4, 6

```

#include<stdio.h>
#define MAX 20
int frames[MAX],ref[MAX],mem[MAX][MAX],faults,sp,m,n;

```

```

void accept()
{
    int i;
    printf("Enter no.of frames:");
    scanf("%d", &n);
    printf("Enter no.of references:");
    scanf("%d", &m);
    printf("Enter reference string:\n");
    for(i=0;i<m;i++)
    {
        printf("[%d]=",i);
        scanf("%d",&ref[i]);
    }
}

void disp()
{
    int i,j;
    for(i=0;i<m;i++)
        printf("%3d",ref[i]);
    printf("\n\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<m;j++)
        {
            if(mem[i][j])
                printf("%3d",mem[i][j]);
            else
                printf(" ");
        }
        printf("\n");
    }
    printf("Total Page Faults: %d\n",faults);
}

int search(int pno)
{
    int i;
    for(i=0;i<n;i++)
    {
        if(frames[i]==pno)
            return i;
    }
    return -1;
}

void fifo()
{
    int i,j;
    for(i=0;i<m;i++)
    {

```

```

if(search(ref[i])== -1)
{
frames[sp] = ref[i];
sp = (sp+1)%n;
faults++;
for(j=0;j<n;j++)
mem[j][i] = frames[j];
}
}
}
int main()
{
accept();
fifo();
disp();
return 0;
}

```

Slip 15

Q.1) Write a C program to illustrate the concept of orphan process. Parent process creates a child and terminates before child has finished its task. So child process becomes orphan process. (Use fork(), sleep(), getpid(), getppid)

```

#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
int pid;
pid=getpid();
printf("Current Process ID is : %d\n",pid);
printf("\n[Forking Child Process ... ] \n");
pid=fork();
if(pid < 0)
{
printf("\nProcess can not be created ");
}
else
{
if(pid==0) {
printf("\nChild Process is Sleeping ...");
sleep(5);
printf("\nOrphan Child's Parent ID : %d",getppid());
}
else
{ /* Parent Process */
printf("\nParent Process Completed ...");
}
}
return 0;
}

```

```
}
```

Q.2) Write the program to simulate Preemptive Priority scheduling. The arrival time and first CPU-burst and priority for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time .

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
typedef struct process_info
{
    char pname[20];
    int at, bt, ct, bt1, p;
    struct process_info *next;
}NODE;
int n;
NODE *first, *last;
void accept_info()
{
    NODE *p;
    int i;
    printf("Enter no.of process:");
    scanf("%d", &n);
    for(i=0; i<n; i++)
    {
        p = (NODE*)malloc(sizeof(NODE));
        printf("Enter process name:");
        scanf("%s", p->pname);
        printf("Enter arrival time:");
        scanf("%d", &p->at);
        printf("Enter first CPU burst time:");
        scanf("%d", &p->bt);
        printf("Enter priority:");
        scanf("%d", &p->p);
        p->bt1 = p->bt;
        p->next = NULL;
        if(first==NULL)
            first=p;
        else
            last->next=p;
        last = p;
    }
}
void print_output()
{
    NODE *p;
    float avg_tat=0, avg_wt=0;
    printf("pname\tat\tbt\tp\ttct\tttat\twt\n");
```

```

p = first;
while(p!=NULL)
{
int tat = p->ct-p->at;
int wt = tat-p->bt;
avg_tat+=tat;
avg_wt+=wt;
printf("%s\t%d\t%d\t%d\t%d\t%d\n",
p->pname,p->at,p->bt,p->p,p->ct,tat,wt);
p=p->next;
}
printf("Avg TAT=%f\tAvg WT=%f\n",
avg_tat/n,avg_wt/n);
}
void print_input()
{
NODE *p;
p = first;
printf("pname\tat\tbt\ttp\n");
while(p!=NULL)
{
printf("%s\t%d\t%d\t%d\n",
p->pname,p->at,p->bt1,p->p);
p = p->next;
}
}
void sort()
{
NODE *p,*q;
int t;
char name[20];
p = first;
while(p->next!=NULL)
{
q=p->next;
while(q!=NULL)
{
if(p->at > q->at)
{
strcpy(name,p->pname);
strcpy(p->pname,q->pname);
strcpy(q->pname,name);
t = p->at;
p->at = q->at;
q->at = t;
t = p->bt;
p->bt = q->bt;
q->bt = t;
}
}
}
}

```

```

t = p->ct;
p->ct = q->ct;
q->ct = t;
t = p->bt1;
p->bt1 = q->bt1;
q->bt1 = t;
t = p->p;
p->p = q->p;
q->p = t;
}
q=q->next;
}
p=p->next;
}
}
int time;
NODE * get_p()
{
    NODE *p,*min_p=NULL;
    int min=9999;
    p = first;
    while(p!=NULL)
    {
        if(p->at<=time && p->bt1!=0 &&
        p->p<min)
        {
            min = p->p;
            min_p = p;
        }
        p=p->next;
    }
    return min_p;
}
struct gantt_chart
{
    int start;
    char pname[30];
    int end;
    }s[100],s1[100];
    int k;
    void pnp()
    {
        int prev=0,n1=0;
        NODE *p;
        while(n1!=n)
        {
            p = get_p();
            if(p==NULL)

```

```

{
time++;
s[k].start = prev;
strcpy(s[k].pname, "");
s[k].end = time;
prev = time;
k++;
}
else
{
time++;
s[k].start = prev;
strcpy(s[k].pname, p->pname);
s[k].end = time;
prev = time;
k++;
p->ct = time;
p->bt1--;
if(p->bt1==0) n1++;
}
print_input();
sort();
}
}
void print_gantt_chart()
{
int i,j,m;
s1[0] = s[0];
for(i=1,j=0;i<k;i++)
{
if(strcmp(s[i].pname,s1[j].pname)==0)
s1[j].end = s[i].end;
else
s1[++j] = s[i];
}
printf("%d",s1[0].start);
for(i=0;i<=j;i++)
{
m = (s1[i].end - s1[i].start);
for(k=0;k<m/2;k++)
printf("-");
printf("%s",s1[i].pname);
for(k=0;k<(m+1)/2;k++)
printf("-");
printf("%d",s1[i].end);
}
}
}
int main()

```

```

{
accept_info();
sort();
pnp();
print_output();
print_gantt_chart();
return 0;
}

```

Slip 16

Q.1) Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LRU (using counter method) page replacement algorithm. Assume the memory of n frames.

Reference String : 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2

```

#include<stdio.h>
#define MAX 20
int frames[MAX],ref[MAX],mem[MAX][MAX],faults,
sp,m,n,time[MAX];
void accept()
{
int i;
printf("Enter no.of frames:");
scanf("%d", &n);
printf("Enter no.of references:");
scanf("%d", &m);
printf("Enter reference string:\n");
for(i=0;i<m;i++)
{
printf("[%d]=",i);
scanf("%d",&ref[i]);
}
}
void disp()
{
int i,j;
for(i=0;i<m;i++)
printf("%3d",ref[i]);
printf("\n\n");
for(i=0;i<n;i++)
{
for(j=0;j<m;j++)
{
if(mem[i][j])
printf("%3d",mem[i][j]);
else
printf(" ");
}
printf("\n");
}
}

```

```

printf("Total Page Faults: %d\n",faults);
}
int search(int pno)
{
int i;
for(i=0;i<n;i++)
{
if(frames[i]==pno)
return i;
}
return -1;
}
int get_lru()
{
int i,min_i,min=9999;
for(i=0;i<n;i++)
{
if(time[i]<min)
{
min = time[i];
min_i = i;
}
}
return min_i;
}
void lru()
{
int i,j,k;
for(i=0;i<m && sp<n;i++)
{
k=search(ref[i]);
if(k!=-1)
{
frames[sp]=ref[i];
time[sp]=i;
faults++;
sp++;
for(j=0;j<n;j++)
mem[j][i]=frames[j];
}
else
time[k]=i;
}
for(;i<m;i++)
{
k = search(ref[i]);
if(k!=-1)
{

```

```

sp = get_lru();
frames[sp] = ref[i];
time[sp] = i;
faults++;
for(j=0;j<n;j++)
mem[j][i] = frames[j];
}
else
time[k]=i;
}
}
int main()
{
accept();
lru();
disp();
return 0;
}

```

Q.2) Write a C program that behaves like a shell which displays the command prompt '\$'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'count' as

a \$ count c filename: It will display the number of characters in given file

b \$ count w filename: It will display the number of words in given file

c \$ count l filename: It will display the number of lines in given file

```

#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
void make_toks(char *s, char *tok[])
{
int i=0;
char *p;
p = strtok(s, " ");
while(p!=NULL)
{
tok[i++]=p;
p=strtok(NULL, " ");
}
tok[i]=NULL;
}
void count(char *fn, char op)
{
int fh,cc=0,wc=0,lc=0;
char c;

```

```

fh = open(fn,O_RDONLY);
if(fh== -1)
{
printf("File %s not found.\n",fn);
return;
}
while(read(fh,&c,1)>0)
{
if(c==' ') wc++;
else if(c=='\n')
{
wc++;
lc++;
}
cc++;
}
close(fh);
switch(op)
{
case 'c':
printf("No.of characters:%d\n",cc-1);
break;
case 'w':
printf("No.of words:%d\n",wc);
break;
case 'l':
printf("No.of lines:%d\n",lc+1);
break;
}
}
int main()
{
char buff[80],*args[10];
int pid;
while(1)
{
printf("myshell$ ");
fflush(stdin);
fgets(buff,80,stdin);
buff[strlen(buff)-1]='\0';
make_toks(buff,args);
if(strcmp(args[0],"count")==0)
count(args[2],args[1][0]);
else
{
pid = fork();
if(pid>0)
wait();
}
}
}

```

```

else
{
if(execcvp(args[0],args)==-1)
printf("Bad command.\n");
}
}
}
return 0;
}

```

Slip 17

Q.1) Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
typedef struct process_info
{
char pname[20];
int at, bt, ct, bt1;
struct process_info *next;
}NODE;
int n;
NODE *first, *last;
void accept_info()
{
NODE *p;
int i;
printf("Enter no.of process:");
scanf("%d", &n);
for(i=0; i<n; i++)
{
p = (NODE*)malloc(sizeof(NODE));
printf("Enter process name:");
scanf("%s", p->pname);
printf("Enter arrival time:");
scanf("%d", &p->at);
printf("Enter first CPU burst time:");
scanf("%d", &p->bt);
p->bt1 = p->bt;
p->next = NULL;
if(first==NULL)
first=p;
else
last->next=p;
}
}

```

```

last = p;
}
}
void print_output()
{
NODE *p;
float avg_tat=0,avg_wt=0;
printf("pname\tat\tbt\tct\ttat\twt\n");
p = first;
while(p!=NULL)
{
Int tat = p->ct-p->at;
int wt = tat-p->bt;
avg_tat+=tat;
avg_wt+=wt;
printf("%s\t%d\t%d\t%d\t%d\t%d\n",
p->pname,p->at,p->bt,p->ct,tat,wt);
p=p->next;
}
printf("Avg TAT=%f\tAvg WT=%f\n",
avg_tat/n,avg_wt/n);
}
void print_input()
{
NODE *p;
p = first;
printf("pname\tat\tbt\n");
while(p!=NULL)
{
printf("%s\t%d\t%d\n",
p->pname,p->at,p->bt1);
p = p->next;
}
}
void sort()
{
NODE *p,*q;
int t;
char name[20];
p = first;
while(p->next!=NULL)
{
q=p->next;
while(q!=NULL)
{
if(p->at > q->at)
{
strcpy(name,p->pname);

```

```

strcpy(p->pname,q->pname);
strcpy(q->pname,name);
t = p->at;
p->at = q->at;
q->at = t;
t = p->bt;
p->bt = q->bt;
q->bt = t;
t = p->ct;
p->ct = q->ct;
q->ct = t;
t = p->bt1;
p->bt1 = q->bt1;
q->bt1 = t;
}
q=q->next;
}
p=p->next;
}
}
int time;
NODE * get_sjf()
{
NODE *p,*min_p=NULL;
int min=9999;
p = first;
while(p!=NULL)
{
if(p->at<=time && p->bt1!=0 &&
p->bt1<min)
{
min = p->bt1;
min_p = p;
}
p=p->next;
}
return min_p;
}
struct gantt_chart
{
int start;
char pname[30];
int end;
}s[100],s1[100];
int k;
void sjfnp()
{
int prev=0,n1=0;

```

```

NODE *p;
while(n1!=n)
{
p = get_sjf();
if(p==NULL)
{
time++;
s[k].start = prev;
strcpy(s[k].pname,"");
s[k].end = time;
prev = time;
k++;
}
else
{
time+=p->bt1;
s[k].start = prev;
strcpy(s[k].pname, p->pname);
s[k].end = time;
prev = time;
k++;
p->ct = time;
p->bt1 = 0;
n1++;
}
print_input();
sort();
}
}
void print_gantt_chart()
{
int i,j,m;
s1[0] = s[0];
for(i=1,j=0;i<k;i++)
{
if(strcmp(s[i].pname,s1[j].pname)==0)
s1[j].end = s[i].end;
else
s1[++j] = s[i];
}
printf("%d",s1[0].start);
for(i=0;i<=j;i++)
{
m = (s1[i].end - s1[i].start);
for(k=0;k<m/2;k++)
printf("-");
printf("%s",s1[i].pname);
for(k=0;k<(m+1)/2;k++)

```

```

printf("-");
printf("%d",s1[i].end);
}
}
int main()
{
accept_info();
sort();
sjfnp();
print_output();
print_gantt_chart();
return 0;
}

```

Q.2) Write a program to simulate Sequential (Contiguous) file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks. Write menu driven program with menu options as mentioned below and implement each option.

- a. Show Bit Vector**
- b. Create New File**
- c. Show Directory**
- d. Delete File**
- e. Exit**

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#define MAX 200
typedef struct dir
{
char fname[20];
int start;
struct dir *next;
}NODE;
NODE *first,*last;
int n,fb,bit[MAX];
void init()
{
int i;
printf("Enter total no.of disk blocks:");
scanf("%d",&n);
fb = n;
for(i=0;i<10;i++)
{
int k = rand()%n;
if(bit[k]!=-2)
{
bit[k]=-2;

```

```

fb--;
}
}
}
void show_bitvector()
{
int i;
for(i=0;i<n;i++)
printf("%d ",bit[i]);
printf("\n");
}
void show_dir()
{
NODE *p;
int i;
printf("File\tChain\n");
p = first;
while(p!=NULL)
{
printf("%s\t",p->fname);
i = p->start;
while(i!=-1)
{
printf("%d->",i);
i=bit[i];
}
printf("NULL\n");
p=p->next;
}
}
void create()
{
NODE *p;
char fname[20];
int i,j,nob;
printf("Enter file name:");
scanf("%s",fname);
printf("Enter no.of blocks:");
scanf("%d",&nob);
if(nob>fb)
{
printf("Failed to create file %s\n",fname);
return;
}
for(i=0;i<n;i++)
{
if(bit[i]==0) break;
}
}

```

```

p = (NODE*)malloc(sizeof(NODE));
strcpy(p->fname,fname);
p->start=i;
p->next=NULL;
if(first==NULL)
first=p;
else
last->next=p;
last=p;
fb-=nob;
j=i+1;
nob--;
while(nob>0)
{
if(bit[j]==0)
{
bit[i]=j;
i=j;
nob--;
}
j++;
}
bit[i]=- 1;
printf("File %s created successully.\n",fname);
}

void delete()
{
char fname[20];
NODE *p,*q;
int nob=0,i,j;
printf("Enter file name to be deleted:");
scanf("%s",fname);
p = q = first;
while(p!=NULL)
{
if(strcmp(p->fname,fname)==0)
break;
q=p;
p=p->next;
}
if(p==NULL)
{
printf("File %s not found.\n",fname);
return;
}
i = p->start;
while(i!=-1)
{

```

```

nob++;
j = i;
i = bit[i];
bit[j] = 0;
}
fb+=nob;
if(p==first)
first=first->next;
else if(p==last)
{
last=q;
last->next=NULL;
}
else
q->next = p->next;
free(p);
printf("File %s deleted successfully.\n",fname);
}
int main()
{
int ch;
init();
while(1)
{
printf("1.Show bit vector\n");
printf("2.Create new file\n");
printf("3.Show directory\n");
printf("4.Delete file\n");
printf("5.Exit\n");
printf("Enter your choice (1-5):");
scanf("%d",&ch);
switch(ch)
{
case 1:
show_bitvector();
break;
case 2:
create();
break;
case 3:
show_dir();
break;
case 4:
delete();
break;
case 5:
exit(0);
}
}

```

```

}
return 0;
}

```

Slip 18

Q.1) Write the program to simulate Preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU burst for different n number of processes should be input to the algorithm. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
typedef struct process_info
{
char pname[20];
int at, bt, ct, bt1;
struct process_info *next;
}NODE;
int n;
NODE *first, *last;
void accept_info()
{
NODE *p;
int i;
printf("Enter no.of process:");
scanf("%d", &n);
for(i=0; i<n; i++)
{
p = (NODE*)malloc(sizeof(NODE));
printf("Enter process name:");
scanf("%s", p->pname);
printf("Enter arrival time:");
scanf("%d", &p->at);
printf("Enter first CPU burst time:");
scanf("%d", &p->bt);
p->bt1 = p->bt;
p->next = NULL;
if(first==NULL)
first=p;
else
last->next=p;
last = p;
}
}
void print_output()
{
NODE *p;
float avg_tat=0, avg_wt=0;

```

```

printf("pname\tat\tbt\tct\ttat\twt\n");
p = first;
while(p!=NULL)
{
int tat = p->ct-p->at;
int wt = tat-p->bt;
avg_tat+=tat;
avg_wt+=wt;
printf("%s\t%d\t%d\t%d\t%d\t%d\n",
p->pname,p->at,p->bt,p->ct,tat,wt);
p=p->next;
}
printf("Avg TAT=%f\tAvg WT=%f\n",
avg_tat/n,avg_wt/n);
}
void print_input()
{
NODE *p;
p = first;
printf("pname\tat\tbt\n");
while(p!=NULL)
{
printf("%s\t%d\t%d\n",
p->pname,p->at,p->bt1);
p = p->next;
}
}
void sort()
{
NODE *p,*q;
int t;
char name[20];
p = first;
while(p->next!=NULL)
{
q=p->next;
while(q!=NULL)
{
if(p->at > q->at)
{
strcpy(name,p->pname);
strcpy(p->pname,q->pname);
strcpy(q->pname,name);
t = p->at;
p->at = q->at;
q->at = t;
t = p->bt;
p->bt = q->bt;

```

```

q->bt = t;
t = p->ct;
p->ct = q->ct;
q->ct = t;
t = p->bt1;
p->bt1 = q->bt1;
q->bt1 = t;
}
q=q->next;
}
p=p->next;
}
}
int time;
NODE * get_sjf()
{
    NODE *p,*min_p=NULL;
    int min=9999;
    p = first;
    while(p!=NULL)
    {
        if(p->at<=time && p->bt1!=0 &&
        p->bt1<min)
        {
            min = p->bt1;
            min_p = p;
        }
        p=p->next;
    }
    return min_p;
}
struct gantt_chart
{
    int start;
    char pname[30];
    int end;
    }s[100],s1[100];
    int k;
    void sjfp()
    {
        int prev=0,n1=0;
        NODE *p;
        while(n1!=n)
        {
            p = get_sjf();
            if(p==NULL)
            {
                time++;
            }
        }
    }
}

```

```

s[k].start = prev;
strcpy(s[k].pname, "");
s[k].end = time;
prev = time;
k++;
}
else
{
time++;
s[k].start = prev;
strcpy(s[k].pname, p->pname);
s[k].end = time;
prev = time;
k++;
p->ct = time;
p->bt1--;
if(p->bt1==0)
n1++;
}
print_input();
sort();
}
}
void print_gantt_chart()
{
int i,j,m;
s1[0] = s[0];
for(i=1,j=0;i<k;i++)
{
if(strcmp(s[i].pname,s1[j].pname)==0)
s1[j].end = s[i].end;
else
s1[++j] = s[i];
}
printf("%d",s1[0].start);
for(i=0;i<=j;i++)
{
m = (s1[i].end - s1[i].start);
for(k=0;k<m/2;k++)
printf("-");
printf("%s",s1[i].pname);
for(k=0;k<(m+1)/2;k++)
printf("-");
printf("%d",s1[i].end);
}
}
int main()
{

```

```

accept_info();
sort();
sjfp();
print_output();
print_gantt_chart();
return 0;
}

```

Q.2) Write a C program to illustrate the concept of orphan process. Parent process creates a child and terminates before child has finished its task. So child process becomes orphan process. (Use fork(), sleep(), getpid(), getppid())

```

#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
    int pid;
    pid=getpid();
    printf("Current Process ID is : %d\n",pid);
    printf("\n[Forking Child Process ... ] \n");
    pid=fork();
    if(pid < 0)
    {
        printf("\nProcess can not be created ");
    }
    else
    {
        if(pid==0) {
            printf("\nChild Process is Sleeping ...");
            sleep(5);
            printf("\nOrphan Child's Parent ID : %d",getppid());
        }
        else
        { /* Parent Process */
            printf("\nParent Process Completed ...");
        }
    }
    return 0;
}

```

Slip 19

Q.1) Write the program to simulate Preemptive Priority scheduling. The arrival time and first CPU-burst and priority for different n number of processes should be input to the algorithm. The next CPU-burst should be generated.

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
typedef struct process_info
{
    char pname[20];

```

```

int at, bt, ct, bt1, p;
struct process_info *next;
}NODE;
int n;
NODE *first, *last;
void accept_info()
{
    NODE *p;
    int i;
    printf("Enter no.of process:");
    scanf("%d", &n);
    for(i=0; i<n; i++)
    {
        p = (NODE*)malloc(sizeof(NODE));
        printf("Enter process name:");
        scanf("%s", p->pname);
        printf("Enter arrival time:");
        scanf("%d", &p->at);
        printf("Enter first CPU burst time:");
        scanf("%d", &p->bt);
        printf("Enter priority:");
        scanf("%d", &p->p);
        p->bt1 = p->bt;
        p->next = NULL;
        if(first==NULL)
            first=p;
        else
            last->next=p;
        last = p;
    }
}
void print_output()
{
    NODE *p;
    float avg_tat=0, avg_wt=0;
    printf("pname\tat\tbt\tp\tct\ttat\twt\n");
    p = first;
    while(p!=NULL)
    {
        int tat = p->ct-p->at;
        int wt = tat-p->bt;
        avg_tat+=tat;
        avg_wt+=wt;
        printf("%s\t%d\t%d\t%d\t%d\t%d\t%d\n",
            p->pname, p->at, p->bt, p->p, p->ct, tat, wt);
        p=p->next;
    }
    printf("Avg TAT=%f\tAvg WT=%f\n",

```

```

    avg_tat/n,avg_wt/n);
}
void print_input()
{
    NODE *p;
    p = first;
    printf("pname\tat\tbt\tp\n");
    while(p!=NULL)
    {
        printf("%s\t%d\t%d\t%d\n",
            p->pname,p->at,p->bt1,p->p);
        p = p->next;
    }
}
void sort()
{
    NODE *p,*q;
    int t;
    char name[20];
    p = first;
    while(p->next!=NULL)
    {
        q=p->next;
        while(q!=NULL)
        {
            if(p->at > q->at)
            {
                strcpy(name,p->pname);
                strcpy(p->pname,q->pname);
                strcpy(q->pname,name);
                t = p->at;
                p->at = q->at;
                q->at = t;
                t = p->bt;
                p->bt = q->bt;
                q->bt = t;
                t = p->ct;
                p->ct = q->ct;
                q->ct = t;
                t = p->bt1;
                p->bt1 = q->bt1;
                q->bt1 = t;
                t = p->p;
                p->p = q->p;
                q->p = t;
            }
            q=q->next;
        }
    }
}

```

```

p=p->next;
}
}
int time;
NODE * get_p()
{
NODE *p,*min_p=NULL;
int min=9999;
p = first;
while(p!=NULL)
{
if(p->at<=time && p->bt1!=0 &&
p->p<min)
{
min = p->p;
min_p = p;
}
p=p->next;
}
return min_p;
}
struct gantt_chart
{
int start;
char pname[30];
int end;
}s[100],s1[100];
int k;
void pnp()
{
int prev=0,n1=0;
NODE *p;
while(n1!=n)
{
p = get_p();
if(p==NULL)
{
time++;
s[k].start = prev;
strcpy(s[k].pname,"*");
s[k].end = time;
prev = time;
k++;
}
else
{
time++;
s[k].start = prev;

```

```

strcpy(s[k].pname, p->pname);
s[k].end = time;
prev = time;
k++;
p->ct = time;
p->bt1--;
if(p->bt1==0) n1++;
}
print_input();
sort();
}
}
void print_gantt_chart()
{
int i,j,m;
s1[0] = s[0];
for(i=1,j=0;i<k;i++)
{
if(strcmp(s[i].pname,s1[j].pname)==0)
s1[j].end = s[i].end;
else
s1[++j] = s[i];
}
printf("%d",s1[0].start);
for(i=0;i<=j;i++)
{
m = (s1[i].end - s1[i].start);
for(k=0;k<m/2;k++)
printf("-");
printf("%s",s1[i].pname);
for(k=0;k<(m+1)/2;k++)
printf("-");
printf("%d",s1[i].end);
}
}
int main()
{
accept_info();
sort();
pnp();
print_output();
print_gantt_chart();
return 0;
}

```

Q.2) Write a program that demonstrates the use of nice() system call. After a child process is started using fork(), assign higher priority to the child using nice() system call

```
#include <stdio.h>
```

```

#include <sys/types.h>
#include <unistd.h>
int main()
{
    pid_t pid;
    pid = fork();
    if (pid == 0)
    {
        printf("\nI am child process, id=%d\n",getpid());
        printf("\nPriority :%d,id=%d\n",nice (-7),getpid());
    }
    else
    {
        printf("\nI am parent process, id=%d\n",getpid());
        nice(1);
        printf("\nPriority :%d,id=%d\n",nice (15),getpid());
    }
    return 0;
}

```

Slip 20

Q.1) Write a simulation program for disk scheduling using LOOK algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.

86, 147, 91, 177, 45, 12, 130

Starting Head Position: 60

Direction: Right

```

#include<stdio.h>
#include<stdlib.h>
int main()
{
    int RQ[100],i,j,n,TotalHeadMoment=0,initial,size,move;
    printf("Enter the number of Requests\n");
    scanf("%d",&n);
    printf("Enter the Requests sequence\n");
    for(i=0;i<n;i++)
        scanf("%d",&RQ[i]);
    printf("Enter initial head position\n");
    scanf("%d",&initial);
    printf("Enter total disk size\n");
    scanf("%d",&size);
    printf("Enter the head movement direction for high 1 and for low 0\n");
    scanf("%d",&move);
    // logic for look disk scheduling
    /*logic for sort the request array */
    for(i=0;i<n;i++)
    {
        for(j=0;j<n-i-1;j++)

```

```

{
if(RQ[j]>RQ[j+1])
{
int temp;
temp=RQ[j];
RQ[j]=RQ[j+1];
RQ[j+1]=temp;
}
}
}
int index;
for(i=0;i<n;i++)
{
if(initial<RQ[i])
{
index=i;
break;
}
}
// if movement is towards high value
if(move==1)
{
for(i=index;i<n;i++)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
for(i=index-1;i>=0;i--)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
}
// if movement is towards low value
else
{
for(i=index-1;i>=0;i--)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
for(i=index;i<n;i++)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
}
printf("Total head movement is %d",TotalHeadMoment);

```

```
return 0;
}
```

Q.2) Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LRU (using counter method) page replacement algorithm. Assume the memory of n frames.

Reference String : 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2

```
#include<stdio.h>
#define MAX 20
int frames[MAX],ref[MAX],mem[MAX][MAX],faults,
sp,m,n,time[MAX];
void accept()
{
    int i;
    printf("Enter no.of frames:");
    scanf("%d", &n);
    printf("Enter no.of references:");
    scanf("%d", &m);
    printf("Enter reference string:\n");
    for(i=0;i<m;i++)
    {
        printf("[%d]= ",i);
        scanf("%d",&ref[i]);
    }
}
void disp()
{
    int i,j;
    for(i=0;i<m;i++)
        printf("%3d",ref[i]);
    printf("\n\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<m;j++)
        {
            if(mem[i][j])
                printf("%3d",mem[i][j]);
            else
                printf(" ");
        }
        printf("\n");
    }
    printf("Total Page Faults: %d\n",faults);
}
int search(int pno)
{
    int i;
    for(i=0;i<n;i++)
```

```

{
if(frames[i]==pno)
return i;
}
return -1;
}
int get_lru()
{
int i,min_i,min=9999;
for(i=0;i<n;i++)
{
if(time[i]<min)
{
min = time[i];
min_i = i;
}
}
return min_i;
}
void lru()
{
int i,j,k;
for(i=0;i<m && sp<n;i++)
{
k=search(ref[i]);
if(k!=-1)
{
frames[sp]=ref[i];
time[sp]=i;
faults++;
sp++;
for(j=0;j<n;j++)
mem[j][i]=frames[j];
}
else
time[k]=i;
}
for(i<m;i++)
{
k = search(ref[i]);
if(k!=-1)
{
sp = get_lru();
frames[sp] = ref[i];
time[sp] = i;
faults++;
for(j=0;j<n;j++)
mem[j][i] = frames[j];

```

```

}
else
time[k]=i;
}
}
int main()
{
accept();
lru();
disp();
return 0;
}

```

Slip 21

Q.1) Write the simulation program for demand paging and show the page scheduling and total number of page faults according the MFU page replacement algorithm. Assume the memory of n frames.

Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2

```

#include<stdio.h>
#define MAX 20
int frames[MAX],ref[MAX],mem[MAX][MAX],faults,
sp,m,n,count[MAX];
void accept()
{
int i;
printf("Enter no.of frames:");
scanf("%d", &n);
printf("Enter no.of references:");
scanf("%d", &m);
printf("Enter reference string:\n");
for(i=0;i<m;i++)
{
printf("[%d]=",i);
scanf("%d",&ref[i]);
}
}
void disp()
{
int i,j;
for(i=0;i<m;i++)
printf("%3d",ref[i]);
printf("\n\n");
for(i=0;i<n;i++)
{
for(j=0;j<m;j++)
{
if(mem[i][j])
printf("%3d",mem[i][j]);
else

```

```

printf(" ");
}
printf("\n");
}
printf("Total Page Faults: %d\n",faults);
}
int search(int pno)
{
int i;
for(i=0;i<n;i++)
{
if(frames[i]==pno)
return i;
}
return -1;
}
int get_mfu(int sp)
{
int i,max_i,max=-9999;
i=sp;
do
{
if(count[i]>max)
{
max = count[i];
max_i = i;
}
i=(i+1)%n;
}while(i!=sp);
return max_i;
}
void mfu()
{
int i,j,k;
for(i=0;i<m && sp<n;i++)
{
k=search(ref[i]);
if(k!=-1)
{
frames[sp]=ref[i];
count[sp]++;
faults++;
sp++;

for(j=0;j<n;j++)
mem[j][i]=frames[j];
}
else

```

```

        count[k]++;

    }

    sp=0;
    for(;i<m;i++)
    {
        k = search(ref[i]);
        if(k!=-1)
        {
            sp = get_mfu(sp);
            frames[sp] = ref[i];
            count[sp]=1;
            faults++;
            sp = (sp+1)%n;

            for(j=0;j<n;j++)
                mem[j][i] = frames[j];
        }
        else
            count[k]++;
    }
}

int main()
{
    accept();
    mfu();
    disp();
    return 0;
}

```

Q.2) Write a program that demonstrates the use of nice() system call. After a child process is started using fork(), assign higher priority to the child using nice() system call.

```

#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
    pid_t pid;
    pid = fork();
    if (pid == 0)
    {
        printf("\nI am child process, id=%d\n",getpid());
        printf("\nPriority :%d,id=%d\n",nice (-7),getpid());
    }
    else
    {
        printf("\nI am parent process, id=%d\n",getpid());
    }
}

```

```

nice(1);
printf("\nPriority :%d,id=%d\n",nice (15),getpid());
}
return 0;
}

```

Slip 22

Q.1) Write a program to simulate Linked file allocation method. Assume disk with 'n' number of blocks. Give value of 'n' as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driven program with menu options as mentioned below and implement each option.

- a. Show Bit Vector**
- b. Create New File**
- c. Show Directory**
- d. Delete File**
- e. Exit**

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#define MAX 200
typedef struct dir
{
char fname[20];
int start;
struct dir *next;
}NODE;
NODE *first,*last;
int n,fb,bit[MAX];
void init()
{
int i;
printf("Enter total no.of disk blocks:");
scanf("%d",&n);
fb = n;
for(i=0;i<10;i++)
{
int k = rand()%n;
if(bit[k]!=-2)
{
bit[k]=-2;
fb--;
}
}
}
void show_bitvector()
{
int i;
for(i=0;i<n;i++)
printf("%d ",bit[i]);

```

```

printf("\n");
}
void show_dir()
{
    NODE *p;
    int i;
    printf("File\tChain\n");
    p = first;
    while(p!=NULL)
    {
        printf("%s\t",p->fname);
        i = p->start;
        while(i!=-1)
        {
            printf("%d->",i);
            i=bit[i];
        }
        printf("NULL\n");
        p=p->next;
    }
}
void create()
{
    NODE *p;
    char fname[20];
    int i,j,nob;
    printf("Enter file name:");
    scanf("%s",fname);
    printf("Enter no.of blocks:");
    scanf("%d",&nob);
    if(nob>fb)
    {
        printf("Failed to create file %s\n",fname);
        return;
    }
    for(i=0;i<n;i++)
    {
        if(bit[i]==0) break;
    }
    p = (NODE*)malloc(sizeof(NODE));
    strcpy(p->fname,fname);
    p->start=i;
    p->next=NULL;
    if(first==NULL)
        first=p;
    else
        last->next=p;
    last=p;
}

```

```

fb-=nob;
j=i+1;
nob--;
while(nob>0)
{
if(bit[j]==0)
{
bit[i]=j;
i=j;
nob--;
}
j++;
}
bit[i]=-1;
printf("File %s created successully.\n",fname);
}
void delete()
{
char fname[20];
NODE *p,*q;
int nob=0,i,j;
printf("Enter file name to be deleted:");
scanf("%s",fname);
p = q = first;
while(p!=NULL)
{
if(strcmp(p->fname,fname)==0)
break;
q=p;
p=p->next;
}
if(p==NULL)
{
printf("File %s not found.\n",fname);
return;
}
i = p->start;
while(i!=-1)
{
nob++;
j = i;
i = bit[i];
bit[j] = 0;
}
fb+=nob;
if(p==first)
first=first->next;
else if(p==last)

```

```

{
last=q;
last->next=NULL;
}
else
q->next = p->next;
free(p);
printf("File %s deleted successfully.\n",fname);
}
int main()
{
int ch;
init();
while(1)
{
printf("1.Show bit vector\n");
printf("2.Create new file\n");
printf("3.Show directory\n");
printf("4.Delete file\n");
printf("5.Exit\n");
printf("Enter your choice (1-5):");
scanf("%d",&ch);
switch(ch)
{
case 1:
show_bitvector();
break;
case 2:
create();
break;
case 3:
show_dir();
break;
case 4:
delete();
break;
case 5:
exit(0);
}
}
return 0;
}

```

Q.2) Write the program to simulate Non-preemptive Priority scheduling.

The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

#include<stdio.h>

```

#include<stdlib.h>
#include<string.h>
typedef struct process_info
{
char pname[20];
int at, bt, ct, bt1, p;
struct process_info *next;
}NODE;
int n;
NODE *first, *last;
void accept_info()
{
NODE *p;
int i;
printf("Enter no.of process:");
scanf("%d", &n);
for(i=0; i<n; i++)
{
p = (NODE*)malloc(sizeof(NODE));
printf("Enter process name:");
scanf("%s", p->pname);
printf("Enter arrival time:");
scanf("%d", &p->at);
printf("Enter first CPU burst time:");
scanf("%d", &p->bt);
printf("Enter priority:");
scanf("%d", &p->p);
p->bt1 = p->bt;
p->next = NULL;
if(first==NULL)
first=p;
else
last->next=p;
last = p;
}
}
void print_output()
{
NODE *p;
float avg_tat=0, avg_wt=0;
printf("pname\tat\tbt\tp\tttct\tttat\twt\n");
p = first;
while(p!=NULL)
{
int tat = p->ct-p->at;
int wt = tat-p->bt;
avg_tat+=tat;
avg_wt+=wt;

```

```

printf("%s\t%d\t%d\t%d\t%d\t%d\t%d\n",
p->pname,p->at,p->bt,p->p,p->ct,tat,wt);
p=p->next;
}
printf("Avg TAT=%f\tAvg WT=%f\n",
avg_tat/n,avg_wt/n);
}
void print_input()
{
NODE *p;
p = first;
printf("pname\tat\tbt\tp\n");
while(p!=NULL)
{
printf("%s\t%d\t%d\t%d\n",
p->pname,p->at,p->bt1,p->p);
p = p->next;
}
}
void sort()
{
NODE *p,*q;
int t;
char name[20];
p = first;
while(p->next!=NULL)
{
q=p->next;
while(q!=NULL)
{
if(p->at > q->at)
{
strcpy(name,p->pname);
strcpy(p->pname,q->pname);
strcpy(q->pname,name);
t = p->at;
p->at = q->at;
q->at = t;
t = p->bt;
p->bt = q->bt;
q->bt = t;
t = p->ct;
p->ct = q->ct;
q->ct = t;
t = p->bt1;
p->bt1 = q->bt1;
q->bt1 = t;
t = p->p;

```

```

p->p = q->p;
q->p = t;
}
q=q->next;
}
p=p->next;
}
}
int time;
NODE * get_p()
{
NODE *p,*min_p=NULL;
int min=9999;
p = first;
while(p!=NULL)
{
if(p->at<=time && p->bt1!=0 &&
p->p<min)
{
min = p->p;
min_p = p;
}
p=p->next;
}
return min_p;
}
struct gantt_chart
{
int start;
char pname[30];
int end;
}s[100],s1[100];
int k;
void pnp()
{
int prev=0,n1=0;
NODE *p;
while(n1!=n)
{
p = get_p();
if(p==NULL)
{
time++;
s[k].start = prev;
strcpy(s[k].pname,"*");
s[k].end = time;
prev = time;
k++;

```

```

    }
    else
    {
        time+=p->bt1;
        s[k].start = prev;
        strcpy(s[k].pname, p->pname);
        s[k].end = time;
        prev = time;
        k++;
        p->ct = time;
        p->bt1 = 0;
        n1++;
    }
    print_input();
    sort();
}
}

void print_gantt_chart()
{
    int i,j,m;
    s1[0] = s[0];
    for(i=1,j=0;i<k;i++)
    {
        if(strcmp(s[i].pname,s1[j].pname)==0)
            s1[j].end = s[i].end;
        else
            s1[++j] = s[i];
    }
    printf("%d",s1[0].start);
    for(i=0;i<=j;i++)
    {
        m = (s1[i].end - s1[i].start);
        for(k=0;k<m/2;k++)
            printf("-");
        printf("%s",s1[i].pname);
        for(k=0;k<(m+1)/2;k++)
            printf("-");
        printf("%d",s1[i].end);
    }
}

int main()
{
    accept_info();
    sort();
    pnp();
    print_output();
    print_gantt_chart();
    return 0;
}

```

```
}
```

Slip 23

Q.1) Implement the C program to accept n integers to be sorted. Main function creates child process using fork system call. Parent process sorts the integers using bubble sort and waits for child process using wait system call. Child process sorts the integers using insertion sort

```
#include<stdio.h>
#include<sys/types.h>
#include<unistd.h>
#include<stdlib.h>
void bubblesort(int arr[30],int n)
{
    int i,j,temp;
    for(i=0;i<n;i++)
    {
        for(j=0;j<n-1;j++)
        {
            if(arr[j]>arr[j+1])
            {
                temp=arr[j];
                arr[j]=arr[j+1];
                arr[j+1]=temp;
            }
        }
    }
}
void insertionsort(int arr[30], int n)
{
    int i, j, temp;
    for (i = 1; i < n; i++) {
        temp = arr[i];
        j = i - 1;

        while(j>=0 && temp <= arr[j])
        {
            arr[j+1] = arr[j];
            j = j-1;
        }
        arr[j+1] = temp;
    }
}
void fork1()
{
    int arr[30],n,i,status;
    printf("\nEnter the no of values in array :");
    scanf("%d",&n);
    printf("\nEnter the array elements :");
    for(i=0;i<n;i++)
```

```

        scanf("%d",&arr[i]);
int pid=fork();
if(pid==0)
{
    sleep(10);
    printf("\nchild process\n");
    printf("child process id=%d\n",getpid());
    insertionsort(arr,n);
    printf("\nElements Sorted Using insertion sort:");
    printf("\n");
    for(i=0;i<n;i++)
    printf("%d",arr[i]);
    printf("\b");
    printf("\nparent process id=%d\n",getppid());
    system("ps -x");
}
else
{
    printf("\nparent process\n");
    printf("\nparent process id=%d\n",getppid());
    bubblesort(arr,n);
    printf("Elements Sorted Using bubble sort:");
    printf("\n");
    for(i=0;i<n;i++)
    printf("%d",arr[i]);
    printf("\n\n\n");
}
}
int main()
{
    fork1();
    return 0;
}

```

Q.2) Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
typedef struct process_info
{
    char pname[20];
    int at,bt,ct,bt1;
    struct process_info *next;
}NODE;

```

```

int n;
NODE *first,*last;
void accept_info()
{
    NODE *p;
    int i;
    printf("Enter no.of process:");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        p = (NODE*)malloc(sizeof(NODE));
        printf("Enter process name:");
        scanf("%s",p->pname);
        printf("Enter arrival time:");
        scanf("%d",&p->at);
        printf("Enter first CPU burst time:");
        scanf("%d",&p->bt);
        p->bt1 = p->bt;
        p->next = NULL;
        if(first==NULL)
            first=p;
        else
            last->next=p;
        last = p;
    }
}

void print_output()
{
    NODE *p;
    float avg_tat=0,avg_wt=0;
    printf("pname\tat\tbt\tct\ttat\twt\n");
    p = first;
    while(p!=NULL)
    {
        int tat = p->ct-p->at;
        int wt = tat-p->bt;
        avg_tat+=tat;
        avg_wt+=wt;
        printf("%s\t%d\t%d\t%d\t%d\t%d\n",
            p->pname,p->at,p->bt,p->ct,tat,wt);
        p=p->next;
    }
    printf("Avg TAT=%f\tAvg WT=%f\n",
        avg_tat/n,avg_wt/n);
}

void print_input()
{
    NODE *p;

```

```

p = first;
printf("pname\tat\tbt\n");
while(p!=NULL)
{
printf("%s\t%d\t%d\n",
p->pname,p->at,p->bt1);
p = p->next;
}
}
void sort()
{
NODE *p,*q;
int t;
char name[20];
p = first;
while(p->next!=NULL)
{
q=p->next;
while(q!=NULL)
{
if(p->at > q->at)
{
strcpy(name,p->pname);
strcpy(p->pname,q->pname);
strcpy(q->pname,name);
t = p->at;
p->at = q->at;
q->at = t;
t = p->bt;
p->bt = q->bt;
q->bt = t;
t = p->ct;
p->ct = q->ct;
q->ct = t;
t = p->bt1;
p->bt1 = q->bt1;
q->bt1 = t;
}
q=q->next;
}
p=p->next;
}
}
int time;
NODE * get_sjf()
{
NODE *p,*min_p=NULL;
int min=9999;

```

```

p = first;
while(p!=NULL)
{
if(p->at<=time && p->bt1!=0 &&
p->bt1<min)
{
min = p->bt1;
min_p = p;
}
p=p->next;
}
return min_p;
}

struct gantt_chart
{
int start;
char pname[30];
int end;
}s[100],s1[100];
int k;
void sjfnp()
{
int prev=0,n1=0;
NODE *p;
while(n1!=n)
{
p = get_sjf();
if(p==NULL)
{
time++;
s[k].start = prev;
strcpy(s[k].pname,"");
s[k].end = time;
prev = time;
k++;
}
else
{
time+=p->bt1;
s[k].start = prev;
strcpy(s[k].pname, p->pname);
s[k].end = time;
prev = time;
k++;
p->ct = time;
p->bt1 = 0;
n1++;
}
}

```

```

print_input();
sort();
}
}
void print_gantt_chart()
{
int i,j,m;
s1[0] = s[0];
for(i=1,j=0;i<k;i++)
{
if(strcmp(s[i].pname,s1[j].pname)==0)
s1[j].end = s[i].end;
else
s1[++j] = s[i];
}
printf("%d",s1[0].start);
for(i=0;i<=j;i++)
{
m = (s1[i].end - s1[i].start);
for(k=0;k<m/2;k++)
printf("-");
printf("%s",s1[i].pname);
for(k=0;k<(m+1)/2;k++)
printf("-");
printf("%d",s1[i].end);
}
}
int main()
{
accept_info();
sort();
sjfnp();
print_output();
print_gantt_chart();
return 0;
}

```

Slip 24

Q.1) Write the program to simulate Round Robin (RR) scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. Also give the time quantum as input. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
typedef struct process_info
{

```

```

char pname[20];
int at, bt, ct, bt1;
struct process_info *next;
}NODE;
int n, ts;
NODE *first, *last;
void accept_info()
{
    NODE *p;
    int i;
    printf("Enter no.of process:");
    scanf("%d", &n);
    for(i=0; i<n; i++)
    {
        p = (NODE*)malloc(sizeof(NODE));
        printf("Enter process name:");
        scanf("%s", p->pname);
        printf("Enter arrival time:");
        scanf("%d", &p->at);
        printf("Enter first CPU burst time:");
        scanf("%d", &p->bt);
        p->bt1 = p->bt;
        p->next = NULL;
        if(first==NULL)
            first=p;
        else
            last->next=p;
        last = p;
    }
    printf("Enter time slice:");
    scanf("%d", &ts);
}
void print_output()
{
    NODE *p;
    float avg_tat=0, avg_wt=0;
    printf("pname\tat\tbt\tct\ttat\twt\n");
    p = first;
    while(p!=NULL)
    {
        int tat = p->ct-p->at;
        int wt = tat-p->bt;
        avg_tat+=tat;
        avg_wt+=wt;
        printf("%s\t%d\t%d\t%d\t%d\t%d\n",
            p->pname, p->at, p->bt, p->ct, tat, wt);
        p=p->next;
    }
}

```

```

printf("Avg TAT=%f\tAvg WT=%f\n",
avg_tat/n,avg_wt/n);
}
void print_input()
{
NODE *p;
p = first;
printf("pname\tat\tbt\n");
while(p!=NULL)
{
printf("%s\t%d\t%d\n",
p->pname,p->at,p->bt1);
p = p->next;
}
}
void sort()
{
NODE *p,*q;
int t;
char name[20];
p = first;
while(p->next!=NULL)
{
q=p->next;
while(q!=NULL)
{
if(p->at > q->at)
{
strcpy(name,p->pname);
strcpy(p->pname,q->pname);
strcpy(q->pname,name);
t = p->at;
p->at = q->at;
q->at = t;
t = p->bt;
p->bt = q->bt;
q->bt = t;
t = p->ct;
p->ct = q->ct;
q->ct = t;
t = p->bt1;
p->bt1 = q->bt1;
q->bt1 = t;
}
q=q->next;
}
p=p->next;
}
}

```

```

}
int time;
int is_arrived()
{
    NODE *p;
    p = first;
    while(p!=NULL)
    {
        if(p->at<=time && p->bt1!=0)
            return 1;
        p=p->next;
    }
    return 0;
}
NODE * delq()
{
    NODE *t;
    t = first;
    first = first->next;
    t->next=NULL;
    return t;
}
void addq(NODE *t)
{
    last->next = t;
    last = t;
}
struct gantt_chart
{
    int start;
    char pname[30];
    int end;
}s[100],s1[100];
int k;
void rr()
{
    int prev=0,n1=0;
    NODE *p;
    while(n1!=n)
    {
        if(!is_arrived())
        {
            time++;
            s[k].start = prev;
            strcpy(s[k].pname,"*");
            s[k].end = time;
            k++;
            prev=time;

```

```

    }
    else
    {
        p = first;
        while(1)
        {
            if(p->at<=time && p->bt1!=0)
                break;
            p = delq();
            addq(p);
            p = first;
        }
        if(p->bt1<=ts)
        {
            time+=p->bt1;
            p->bt1=0;
        }
        else
        {
            time+=ts;
            p->bt1-=ts;
        }
        p->ct = time;
        s[k].start = prev;
        strcpy(s[k].pname,p->pname);
        s[k].end = time;
        k++;
        prev = time;
        if(p->bt1==0) n1++;
        p = delq();
        addq(p);
    }
    print_input();
}

void print_gantt_chart()
{
    int i,j,m;
    s1[0] = s[0];
    for(i=1,j=0;i<k;i++)
    {
        if(strcmp(s[i].pname,s1[j].pname)==0)
            s1[j].end = s[i].end;
        else
            s1[++j] = s[i];
    }
    printf("%d",s1[0].start);
    for(i=0;i<=j;i++)

```

```

{
m = (s1[i].end - s1[i].start);
for(k=0;k<m/2;k++)
printf("-");
printf("%s",s1[i].pname);
for(k=0;k<(m+1)/2;k++)
printf("-");
printf("%d",s1[i].end);
}
}
int main()
{
accept_info();
sort();
rr();
print_output();
print_gantt_chart();
return 0;
}

```

Q.2) Implement the C program that accepts an integer array. Main function forks child process. Parent process sorts an integer array and passes the sorted array to child process through the command line arguments of `execve()` system call. The child process uses `execve()` system call to load new program that uses this sorted array for performing the binary search the particular item in the array.

```

//parent.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void bubbleSort(int arr[], int n)
{
    int i, j, temp;
    for(i = 0; i < n - 1; i++)
    {
        for(j = 0; j < n - i - 1; j++)
        {
            if(arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

```

```

int main()
{
    int n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter array elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    bubbleSort(arr, n);

    pid_t pid = fork();

    if(pid > 0)
    {
        wait(NULL);

        char **args = (char **)malloc((n + 2) * sizeof(char *));

        args[0] = "./child";

        for(i = 0; i < n; i++)
        {
            args[i + 1] = (char *)malloc(20);
            sprintf(args[i + 1], "%d", arr[i]);
        }

        args[n + 1] = NULL;

        execve("./child", args, NULL);

        perror("execve failed");
    }
    else if(pid == 0)
    {
        exit(0);
    }

    return 0;
}
//child.c
#include <stdio.h>
#include <stdlib.h>
int binarySearch(int arr[], int n, int key)
{

```

```

int low = 0, high = n - 1;

while(low <= high)
{
    int mid = (low + high) / 2;

    if(arr[mid] == key)
        return mid;
    else if(arr[mid] < key)
        low = mid + 1;
    else
        high = mid - 1;
}

return -1;
}

int main(int argc, char *argv[])
{
    int n = argc - 1;
    int arr[n];
    for(int i = 1; i < argc; i++)
        arr[i - 1] = atoi(argv[i]);
    int key;
    printf("Sorted Array: ");
    for(int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\nEnter element to search: ");
    scanf("%d", &key);
    int pos = binarySearch(arr, n, key);
    if(pos != -1)
        printf("Element found at position %d\n", pos + 1);
    else
        printf("Element not found\n");
    return 0;
}

```

Slip 25

Q.1) Write a C program to illustrate the concept of orphan process. Parent process creates a child and terminates before child has finished its task. So child process becomes orphan process. (Use fork(), sleep(), getpid(), getppid())

```

#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
    int pid;
    pid=getpid();
    printf("Current Process ID is : %d\n",pid);
}

```

```

printf("\n[Forking Child Process ... ] \n");
pid=fork();
if(pid < 0)
{
printf("\nProcess can not be created ");
}
else
{
if(pid==0) {
printf("\nChild Process is Sleeping ...");
sleep(5);
printf("\nOrphan Child's Parent ID : %d",getppid());
}
else
{ /* Parent Process */
printf("\nParent Process Completed ...");
}
}
return 0;
}

```

Q.2) Write a C program that behaves like a shell which displays the command prompt '\$'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'search' as

- a. \$ search f filename pattern : It will search the first occurrence of pattern in the given file**
- b. \$ search a filename pattern : It will search all the occurrence of pattern in the given file**
- c. \$ search c filename pattern : It will count the number of occurrence of pattern in the given file**

```

#include<sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
void make_toks(char *s, char *tok[])
{
int i=0;
char *p;
p = strtok(s, " ");
while(p!=NULL)
{
tok[i++]=p;
p=strtok(NULL, " ");
}
}

```

```

tok[i]=NULL;
}
void search(char *fn, char op, char *pattern)
{
int fh,count=0,i=0,j=0;
char buff[255],c,*p;
fh = open(fn,O_RDONLY);
if(fh== -1)
{
printf("File %s Not Found\n",fn);
return;
}
switch(op)
{
case 'f':
while(read(fh,&c,1))
{
buff[j++]=c;
if(c=='\n')
{
buff[j]='\0';
j=0;
i++;
if(strstr(buff,pattern))
{
printf("%d: %s",i,buff);
break;
}
}
}
break;
case 'c':
while(read(fh,&c,1))
{
buff[j++]=c;
if(c=='\n')
{
buff[j]='\0';
j=0;
p = buff;
while(p=strstr(p,pattern))
{
count++;
p++;
}
}
}
printf("Total No.of Occurrences = %d\n",count);

```

```

break;
case 'a':
while(read(fh,&c,1))
{
buff[j++]=c;
if(c=='\n')
{
buff[j]='\0';
j = 0;
i++;
if(strstr(buff,pattern))
printf("%d: %s",i,buff);
}
}
} //switch
close(fh);
} //search
int main()
{
char buff[80],*args[10];
int pid;
while(1)
{
printf("myshell$");
fflush(stdin);
fgets(buff,80,stdin);
buff[strlen(buff)-1]='\0';
make_toks(buff,args);
if(strcmp(args[0],"search")==0)
search(args[3],args[1][0],args[2]);
else
{
pid = fork();
if(pid>0)
wait();
else
{
if(execvp(args[0],args)==-1)
printf("Bad command.\n");
}
}
}
return 0;
}

```

Slip 26

Q.1) Write the program to simulate Preemptive Priority scheduling. The arrival time and first CPU-burst and priority for different n number of processes should be input to the algorithm. The next CPU-burst should be generated

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
typedef struct process_info
{
char pname[20];
int at, bt, ct, bt1, p;
struct process_info *next;
}NODE;
int n;
NODE *first, *last;
void accept_info()
{
NODE *p;
int i;
printf("Enter no.of process:");
scanf("%d", &n);
for(i=0; i<n; i++)
{
p = (NODE*)malloc(sizeof(NODE));
printf("Enter process name:");
scanf("%s", p->pname);
printf("Enter arrival time:");
scanf("%d", &p->at);
printf("Enter first CPU burst time:");
scanf("%d", &p->bt);
printf("Enter priority:");
scanf("%d", &p->p);
p->bt1 = p->bt;
p->next = NULL;
if(first==NULL)
first=p;
else
last->next=p;
last = p;
}
}
void print_output()
{
NODE *p;
float avg_tat=0, avg_wt=0;
printf("pname\tat\tbt\tp\tttct\tttat\twt\n");
p = first;
while(p!=NULL)
{
int tat = p->ct-p->at;
int wt = tat-p->bt;
avg_tat+=tat;

```

```

avg_wt+=wt;
printf("%s\t%d\t%d\t%d\t%d\t%d\n",
p->pname,p->at,p->bt,p->p,p->ct,tat,wt);
p=p->next;
}
printf("Avg TAT=%f\tAvg WT=%f\n",
avg_tat/n,avg_wt/n);
}
void print_input()
{
NODE *p;
p = first;
printf("pname\tat\tbt\tp\n");
while(p!=NULL)
{
printf("%s\t%d\t%d\t%d\n",
p->pname,p->at,p->bt1,p->p);
p = p->next;
}
}
void sort()
{
NODE *p,*q;
int t;
char name[20];
p = first;
while(p->next!=NULL)
{
q=p->next;
while(q!=NULL)
{
if(p->at > q->at)
{
strcpy(name,p->pname);
strcpy(p->pname,q->pname);
strcpy(q->pname,name);
t = p->at;
p->at = q->at;
q->at = t;
t = p->bt;
p->bt = q->bt;
q->bt = t;
t = p->ct;
p->ct = q->ct;
q->ct = t;
t = p->bt1;
p->bt1 = q->bt1;
q->bt1 = t;

```

```

t = p->p;
p->p = q->p;
q->p = t;
}
q=q->next;
}
p=p->next;
}
}
int time;
NODE * get_p()
{
NODE *p,*min_p=NULL;
int min=9999;
p = first;
while(p!=NULL)
{
if(p->at<=time && p->bt1!=0 &&
p->p<min)
{
min = p->p;
min_p = p;
}
p=p->next;
}
return min_p;
}
struct gantt_chart
{
int start;
char pname[30];
int end;
}s[100],s1[100];
int k;
void pnp()
{
int prev=0,n1=0;
NODE *p;
while(n1!=n)
{
p = get_p();
if(p==NULL)
{
time++;
s[k].start = prev;
strcpy(s[k].pname,"");
s[k].end = time;
prev = time;

```

```

k++;
}
else
{
time++;
s[k].start = prev;
strcpy(s[k].pname, p->pname);
s[k].end = time;
prev = time;
k++;
p->ct = time;
p->bt1--;
if(p->bt1==0) n1++;
}
print_input();
sort();
}
}
void print_gantt_chart()
{
int i,j,m;
s1[0] = s[0];
for(i=1,j=0;i<k;i++)
{
if(strcmp(s[i].pname,s1[j].pname)==0)
s1[j].end = s[i].end;
else
s1[++j] = s[i];
}
printf("%d",s1[0].start);
for(i=0;i<=j;i++)
{
m = (s1[i].end - s1[i].start);
for(k=0;k<m/2;k++)
printf("-");
printf("%s",s1[i].pname);
for(k=0;k<(m+1)/2;k++)
printf("-");
printf("%d",s1[i].end);
}
}
int main()
{
accept_info();
sort();
pnp();
print_output();
print_gantt_chart();

```

```
return 0;
}
```

Q.2) Write the simulation program for demand paging and show the page scheduling and total number of page faults according the FIFO page replacement algorithm.

Assume the memory of n frames.

Reference String: 3, 4, 5, 6, 3, 4, 7, 3, 4, 5, 6, 7, 2, 4, 6

```
#include<stdio.h>
#define MAX 20
int frames[MAX],ref[MAX],mem[MAX][MAX],faults,sp,m,n;
void accept()
{
    int i;
    printf("Enter no.of frames:");
    scanf("%d", &n);
    printf("Enter no.of references:");
    scanf("%d", &m);
    printf("Enter reference string:\n");
    for(i=0;i<m;i++)
    {
        printf("[%d]=",i);
        scanf("%d",&ref[i]);
    }
}
void disp()
{
    int i,j;
    for(i=0;i<m;i++)
        printf("%3d",ref[i]);
    printf("\n\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<m;j++)
        {
            if(mem[i][j])
                printf("%3d",mem[i][j]);
            else
                printf(" ");
        }
        printf("\n");
    }
    printf("Total Page Faults: %d\n",faults);
}
int search(int pno)
{
    int i;
    for(i=0;i<n;i++)
    {
        if(frames[i]==pno)
```

```

return i;
}
return -1;
}
void fifo()
{
int i,j;
for(i=0;i<m;i++)
{
if(search(ref[i])!=-1)
{
frames[sp] = ref[i];
sp = (sp+1)%n;
faults++;
for(j=0;j<n;j++)
mem[j][i] = frames[j];
}
}
}
int main()
{
accept();
fifo();
disp();
return 0;
}

```

Slip 27

Q.1 Write a simulation program for disk scheduling using SSTF algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.

30, 10, 60, 95, 120, 150, 175

Start Head Position: 50

```

#include<stdio.h>
#include<stdlib.h>
int main()
{
int RQ[100],i,n,TotalHeadMoment=0,initial,count=0;
printf("Enter the number of Requests\n");
scanf("%d",&n);
printf("Enter the Requests sequence\n");
for(i=0;i<n;i++)
scanf("%d",&RQ[i]);
printf("Enter initial head position\n");
scanf("%d",&initial);
// logic for sstf disk scheduling
/* loop will execute until all process is completed*/
while(count!=n)

```

```

{
int min=1000,d,index;
for(i=0;i<n;i++)
{
d=abs(RQ[i]-initial);
if(min>d)
{
min=d;
index=i;
}
}
TotalHeadMoment=TotalHeadMoment+min;
initial=RQ[index];
// 1000 is for max
// you can use any number
RQ[index]=1000;
count++;
}
printf("Total head movement is %d",TotalHeadMoment);
printf("\n");
return 0;
}

```

Q.2) Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LRU (using counter method) page replacement algorithm. Assume the memory of n frames.

Reference String : 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2

```

#include<stdio.h>
#define MAX 20
int frames[MAX],ref[MAX],mem[MAX][MAX],faults,
sp,m,n,time[MAX];
void accept()
{
int i;
printf("Enter no.of frames:");
scanf("%d", &n);
printf("Enter no.of references:");
scanf("%d", &m);
printf("Enter reference string:\n");
for(i=0;i<m;i++)
{
printf("[%d]=",i);
scanf("%d",&ref[i]);
}
}
void disp()
{
int i,j;

```

```

for(i=0;i<m;i++)
printf("%3d",ref[i]);
printf("\n\n");
for(i=0;i<n;i++)
{
for(j=0;j<m;j++)
{
if(mem[i][j])
printf("%3d",mem[i][j]);
else
printf(" ");
}
printf("\n");
}
printf("Total Page Faults: %d\n",faults);
}
int search(int pno)
{
int i;
for(i=0;i<n;i++)
{
if(frames[i]==pno)
return i;
}
return -1;
}
int get_lru()
{
int i,min_i,min=9999;
for(i=0;i<n;i++)
{
if(time[i]<min)
{
min = time[i];
min_i = i;
}
}
return min_i;
}
void lru()
{
int i,j,k;
for(i=0;i<m && sp<n;i++)
{
k=search(ref[i]);
if(k==-1)
{
frames[sp]=ref[i];

```

```

time[sp]=i;
faults++;
sp++;
for(j=0;j<n;j++)
mem[j][i]=frames[j];
}
else
time[k]=i;
}
for(;i<m;i++)
{
k = search(ref[i]);
if(k!=-1)
{
sp = get_lru();
frames[sp] = ref[i];
time[sp] = i;
faults++;
for(j=0;j<n;j++)
mem[j][i] = frames[j];
}
else
time[k]=i;
}
}
int main()
{
accept();
lru();
disp();
return 0;
}

```

Slip 28

Q.1 Write a simulation program for disk scheduling using SCAN algorithm.

Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.

10, 25, 75, 90, 130, 145, 180, 55

Starting Head position= 80

Direction: Left

```

#include<stdio.h>
#include<stdlib.h>
int main()
{
int RQ[100],i,j,n,TotalHeadMoment=0,initial,size,move;
printf("Enter the number of Requests\n");
scanf("%d",&n);
printf("Enter the Requests sequence\n");

```

```

for(i=0;i<n;i++)
scanf("%d",&RQ[i]);
printf("Enter initial head position\n");
scanf("%d",&initial);
printf("Enter total disk size\n");
scanf("%d",&size);
printf("Enter the head movement direction for high 1 and for low 0\n");
scanf("%d",&move);
// logic for Scan disk scheduling
/*logic for sort the request array */
for(i=0;i<n;i++)
{
for(j=0;j<n-i-1;j++)
{
if(RQ[j]>RQ[j+1])
{
int temp;
temp=RQ[j];
RQ[j]=RQ[j+1];
RQ[j+1]=temp;
}
}
}
int index;
for(i=0;i<n;i++)
{
if(initial<RQ[i])
{
index=i;
break;
}
}
// if movement is towards high value
if(move==1)
{
for(i=index;i<n;i++)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
// last movement for max size
TotalHeadMoment=TotalHeadMoment+abs(size-RQ[i-1]-1);
initial = size-1;
for(i=index-1;i>=0;i--)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
}

```

```

}
// if movement is towards low value
else
{
for(i=index-1;i>=0;i--)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
// last movement for min size
TotalHeadMoment=TotalHeadMoment+abs(RQ[i+1]-0);
initial =0;
for(i=index;i<n;i++)
{
TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
initial=RQ[i];
}
}
printf("Total head movement is %d",TotalHeadMoment);
return 0;
}

```

Q.2) Write the simulation program for demand paging and show the page scheduling and total number of page faults according the MFU page replacement algorithm. Assume the memory of n frames.

Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2

```

#include<stdio.h>
#define MAX 20
int frames[MAX],ref[MAX],mem[MAX][MAX],faults,
sp,m,n,count[MAX];
void accept()
{
int i;
printf("Enter no.of frames:");
scanf("%d", &n);
printf("Enter no.of references:");
scanf("%d", &m);
printf("Enter reference string:\n");
for(i=0;i<m;i++)
{
printf("[%d]=",i);
scanf("%d",&ref[i]);
}
}
void disp()
{
int i,j;
for(i=0;i<m;i++)
printf("%3d",ref[i]);

```

```

printf("\n\n");
for(i=0;i<n;i++)
{
for(j=0;j<m;j++)
{
if(mem[i][j])
printf("%3d",mem[i][j]);
else
printf(" ");
}
printf("\n");
}
printf("Total Page Faults: %d\n",faults);
}
int search(int pno)
{
int i;
for(i=0;i<n;i++)
{
if(frames[i]==pno)
return i;
}
return -1;
}
int get_mfu(int sp)
{
int i,max_i,max=-9999;
i=sp;
do
{
if(count[i]>max)
{
max = count[i];
max_i = i;
}
i=(i+1)%n;
}while(i!=sp);
return max_i;
}
void mfu()
{
int i,j,k;
for(i=0;i<m && sp<n;i++)
{
k=search(ref[i]);
if(k!=-1)
{
frames[sp]=ref[i];

```

```

count[sp]++;
faults++;
sp++;

for(j=0;j<n;j++)
    mem[j][i]=frames[j];
}
else
    count[k]++;

}

sp=0;
for(;i<m;i++)
{
    k = search(ref[i]);
    if(k!=-1)
    {
        sp = get_mfu(sp);
        frames[sp] = ref[i];
        count[sp]=1;
        faults++;
        sp = (sp+1)%n;

        for(j=0;j<n;j++)
            mem[j][i] = frames[j];
        }
    else
        count[k]++;
    }
}
int main()
{
    accept();
    mfu();
    disp();
    return 0;
}

```

Slip 29

Q.1) Implement the C program that accepts an integer array. Main function forks child process. Parent process sorts an integer array and passes the sorted array to child process through the command line arguments of `execve()` system call. The child process uses `execve()` system call to load new program that uses this sorted array for performing the binary search the particular item in the array

```

//parent.c
#include <stdio.h>
#include <stdlib.h>

```

```

#include <unistd.h>
#include <sys/wait.h>

void bubbleSort(int arr[], int n)
{
    int i, j, temp;
    for(i = 0; i < n - 1; i++)
    {
        for(j = 0; j < n - i - 1; j++)
        {
            if(arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main()
{
    int n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter array elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    bubbleSort(arr, n);

    pid_t pid = fork();

    if(pid > 0)
    {
        wait(NULL);

        char **args = (char **)malloc((n + 2) * sizeof(char *));

        args[0] = "./child";

        for(i = 0; i < n; i++)
        {
            args[i + 1] = (char *)malloc(20);
            sprintf(args[i + 1], "%d", arr[i]);
        }
    }
}

```

```

        args[n + 1] = NULL;

        execve("./child", args, NULL);

        perror("execve failed");
    }
    else if(pid == 0)
    {
        exit(0);
    }

    return 0;
}
//child.c
#include <stdio.h>
#include <stdlib.h>

int binarySearch(int arr[], int n, int key)
{
    int low = 0, high = n - 1;

    while(low <= high)
    {
        int mid = (low + high) / 2;

        if(arr[mid] == key)
            return mid;
        else if(arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}

int main(int argc, char *argv[])
{
    int n = argc - 1;
    int arr[n];
    for(int i = 1; i < argc; i++)
        arr[i - 1] = atoi(argv[i]);
    int key;
    printf("Sorted Array: ");
    for(int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\nEnter element to search: ");
    scanf("%d", &key);
    int pos = binarySearch(arr, n, key);

```

```

if(pos != -1)
printf("Element found at position %d\n", pos + 1);
else
printf("Element not found\n");
return 0;
}

```

Q.2) Write the program to simulate FCFS CPU-scheduling. The arrival time and first CPU- burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
typedef struct process_info
{
char pname[20];
int at,bt,ct,bt1;
struct process_info *next;
}NODE;
int n;
NODE *first,*last;
void accept_info()
{
NODE *p;
int i;
printf("Enter no.of process:");
scanf("%d",&n);
for(i=0;i<n;i++)
{
p = (NODE*)malloc(sizeof(NODE));
printf("Enter process name:");
scanf("%s",p->pname);
printf("Enter arrival time:");
scanf("%d",&p->at);
printf("Enter first CPU burst time:");
scanf("%d",&p->bt);
p->bt1 = p->bt;
p->next = NULL;
if(first==NULL)
first=p;
else
last->next=p;
last = p;
}
}
void print_output()
{

```

```

NODE *p;
float avg_tat=0,avg_wt=0;
printf("pname\tat\tbt\tct\ttat\twt\n");
p = first;
while(p!=NULL)
{
int tat = p->ct-p->at;
int wt = tat-p->bt;
avg_tat+=tat;
avg_wt+=wt;
printf("%s\t%d\t%d\t%d\t%d\t%d\n",
p->pname,p->at,p->bt,p->ct,tat,wt);
p=p->next;
}
printf("Avg TAT=%f\tAvg WT=%f\n",
avg_tat/n,avg_wt/n);
}
void print_input()
{
NODE *p;
p = first;
printf("pname\tat\tbt\n");
while(p!=NULL)
{
printf("%s\t%d\t%d\n",
p->pname,p->at,p->bt1);
p = p->next;
}
}
void sort()
{
NODE *p,*q;
int t;
char name[20];
p = first;
while(p->next!=NULL)
{
q=p->next;
while(q!=NULL)
{
if(p->at > q->at)
{
strcpy(name,p->pname);
strcpy(p->pname,q->pname);
strcpy(q->pname,name);
t = p->at;
p->at = q->at;
q->at = t;

```

```

t = p->bt;
p->bt = q->bt;
q->bt = t;
t = p->ct;
p->ct = q->ct;
q->ct = t;
t = p->bt1;
p->bt1 = q->bt1;
q->bt1 = t;
}
q=q->next;
}
p=p->next;
}
}
int time;
NODE * get_fcfs()
{
    NODE *p;
    p = first;
    while(p!=NULL)
    {
        if(p->at<=time && p->bt1!=0)
            return p;
        p=p->next;
    }
    return NULL;
}
struct gantt_chart
{
    int start;
    char pname[30];
    int end;
}s[100],s1[100];
int k;
void fcfs()
{
    int prev=0,n1=0;
    NODE *p;
    while(n1!=n)
    {
        p = get_fcfs();
        if(p==NULL)
        {
            time++;
            s[k].start = prev;
            strcpy(s[k].pname, "*");
            s[k].end = time;

```

```

prev = time;
k++;
}
else
{
time+=p->bt1;
s[k].start = prev;
strcpy(s[k].pname, p->pname);
s[k].end = time;
prev = time;
k++;
p->ct = time;
p->bt1 = 0;
n1++;
}
print_input();
sort();
}
}
void print_gantt_chart()
{
int i,j,m;
s1[0] = s[0];
for(i=1,j=0;i<k;i++)
{
if(strcmp(s[i].pname,s1[j].pname)==0)
s1[j].end = s[i].end;
else
s1[++j] = s[i];
}
printf("%d",s1[0].start);
for(i=0;i<=j;i++)
{
m = (s1[i].end - s1[i].start);
for(k=0;k<m/2;k++)
printf("-");
printf("%s",s1[i].pname);
for(k=0;k<(m+1)/2;k++)
printf("-");
printf("%d",s1[i].end);
}
}
int main()
{
accept_info();
sort();
fcfs();
print_output();

```

```

print_gantt_chart();
return 0;
}

```

Slip 30

Q.1) Write the program to simulate Round Robin (RR) CPU-scheduling. The arrival time and first CPU- burst for different n number of processes should be input to the algorithm. Also give the time quantum as input. Assume the fixed IO waiting time (2 units). The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
typedef struct process_info
{
    char pname[20];
    int at, bt, ct, bt1;
    struct process_info *next;
}NODE;
int n, ts;
NODE *first, *last;
void accept_info()
{
    NODE *p;
    int i;
    printf("Enter no.of process:");
    scanf("%d", &n);
    for(i=0; i<n; i++)
    {
        p = (NODE*)malloc(sizeof(NODE));
        printf("Enter process name:");
        scanf("%s", p->pname);
        printf("Enter arrival time:");
        scanf("%d", &p->at);
        printf("Enter first CPU burst time:");
        scanf("%d", &p->bt);
        p->bt1 = p->bt;
        p->next = NULL;
        if(first==NULL)
            first=p;
        else
            last->next=p;
        last = p;
    }
    printf("Enter time slice:");
    scanf("%d", &ts);
}
void print_output()

```

```

{
NODE *p;
float avg_tat=0,avg_wt=0;
printf("pname\tat\tbt\tct\ttat\twt\n");
p = first;
while(p!=NULL)
{
int tat = p->ct-p->at;
int wt = tat-p->bt;
avg_tat+=tat;
avg_wt+=wt;
printf("%s\t%d\t%d\t%d\t%d\t%d\n",
p->pname,p->at,p->bt,p->ct,tat,wt);
p=p->next;
}
printf("Avg TAT=%f\tAvg WT=%f\n",
avg_tat/n,avg_wt/n);
}
void print_input()
{
NODE *p;
p = first;
printf("pname\tat\tbt\n");
while(p!=NULL)
{
printf("%s\t%d\t%d\n",
p->pname,p->at,p->bt1);
p = p->next;
}
}
void sort()
{
NODE *p,*q;
int t;
char name[20];
p = first;
while(p->next!=NULL)
{
q=p->next;
while(q!=NULL)
{
if(p->at > q->at)
{
strcpy(name,p->pname);
strcpy(p->pname,q->pname);
strcpy(q->pname,name);
t = p->at;
p->at = q->at;

```

```

q->at = t;
t = p->bt;
p->bt = q->bt;
q->bt = t;
t = p->ct;
p->ct = q->ct;
q->ct = t;
t = p->bt1;
p->bt1 = q->bt1;
q->bt1 = t;
}
q=q->next;
}
p=p->next;
}
}
int time;
int is_arrived()
{
    NODE *p;
    p = first;
    while(p!=NULL)
    {
        if(p->at<=time && p->bt1!=0)
            return 1;
        p=p->next;
    }
    return 0;
}
NODE * delq()
{
    NODE *t;
    t = first;
    first = first->next;
    t->next=NULL;
    return t;
}
void addq(NODE *t)
{
    last->next = t;
    last = t;
}
struct gantt_chart
{
    int start;
    char pname[30];
    int end;
    }s[100],s1[100];

```

```

int k;
void rr()
{
int prev=0,n1=0;
NODE *p;
while(n1!=n)
{
if(!is_arrived())
{
time++;
s[k].start = prev;
strcpy(s[k].pname,"");
s[k].end = time;
k++;
prev=time;
}
else
{
p = first;
while(1)
{
if(p->at<=time && p->bt1!=0)
break;
p = delq();
addq(p);
p = first;
}
if(p->bt1<=ts)
{
time+=p->bt1;
p->bt1=0;
}
else
{
time+=ts;
p->bt1-=ts;
}
p->ct = time;
s[k].start = prev;
strcpy(s[k].pname,p->pname);
s[k].end = time;
k++;
prev = time;
if(p->bt1==0) n1++;
p = delq();
addq(p);
}
print_input();
}

```

```

}
}
void print_gantt_chart()
{
int i,j,m;
s1[0] = s[0];
for(i=1,j=0;i<k;i++)
{
if(strcmp(s[i].pname,s1[j].pname)==0)
s1[j].end = s[i].end;
else
s1[++j] = s[i];
}
printf("%d",s1[0].start);
for(i=0;i<=j;i++)
{
m = (s1[i].end - s1[i].start);
for(k=0;k<m/2;k++)
printf("-");
printf("%s",s1[i].pname);
for(k=0;k<(m+1)/2;k++)
printf("-");
printf("%d",s1[i].end);
}
}
int main()
{
accept_info();
sort();
rr();
print_output();
print_gantt_chart();
return 0;
}

```

Q.2) Write the simulation program for demand paging and show the page scheduling and total number of page faults according the MFU page replacement algorithm. Assume the memory of n frames.

Reference String: 2,5,2,8,5,4,1,2,3,2,6,1,2,5,9,8

```

#include<stdio.h>
#define MAX 20
int frames[MAX],ref[MAX],mem[MAX][MAX],faults,
sp,m,n,count[MAX];
void accept()
{
int i;
printf("Enter no.of frames:");
scanf("%d", &n);
printf("Enter no.of references:");

```

```

scanf("%d", &m);
printf("Enter reference string:\n");
for(i=0;i<m;i++)
{
printf("[%d]=",i);
scanf("%d",&ref[i]);
}
}
void disp()
{
int i,j;
for(i=0;i<m;i++)
printf("%3d",ref[i]);
printf("\n\n");
for(i=0;i<n;i++)
{
for(j=0;j<m;j++)
{
if(mem[i][j])
printf("%3d",mem[i][j]);
else
printf(" ");
}
printf("\n");
}
printf("Total Page Faults: %d\n",faults);
}
int search(int pno)
{
int i;
for(i=0;i<n;i++)
{
if(frames[i]==pno)
return i;
}
return -1;
}
int get_mfu(int sp)
{
int i,max_i,max=-9999;
i=sp;
do
{
if(count[i]>max)
{
max = count[i];
max_i = i;
}
}

```

```

i=(i+1)%n;
}while(i!=sp);
return max_i;
}
void mfu()
{
int i,j,k;
for(i=0;i<m && sp<n;i++)
{
k=search(ref[i]);
if(k!=-1)
{
frames[sp]=ref[i];
count[sp]++;
faults++;
sp++;
for(j=0;j<n;j++)
mem[j][i]=frames[j];
}
else
count[k]++;
}
sp=0;
for(;i<m;i++)
{
k = search(ref[i]);
if(k!=-1)
{
sp = get_mfu(sp);
frames[sp] = ref[i];
count[sp]=1;
faults++;
sp = (sp+1)%n;

for(j=0;j<n;j++)
mem[j][i] = frames[j];
}
else
count[k]++;
}
}
int main()
{
accept();
mfu();
disp();
return 0;
}

```

